

Week10_예습과제

[통계](#) [수정](#) [삭제](#)

카사바 · 방금 전

 0

Euron_2026-1



▼ 목록 보기

6/6



05 토큰화

자연어(Natural Language): 사람들이 쓰는 언어 활동을 위해 자연스럽게 만들어진 언어

자연어 처리(Natural Language Processing, NLP): 컴퓨터가 인간의 언어를 이해, 해석 및 생성하기 위한 기술. 인공지능의 하위 분야 중 하나로 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표 중 하나

인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 모델을 개발하기 위해서는 다음과 같은 문제가 해결되어야 한다:

- 모호성(Ambiguity): 인간 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 한다.
- 가변성(Variability): 인간의 언어는 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 한다.
- 구조(Structure): 인간의 언어는 문장이나 구의 의미를 이해할 때 구문을 파악하여 의미를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하며 의미를 추론하거나 분석할 수 있어야 한다.

위와 같은 문제를 이해하고 구분할 수 있는 모델을 만들려면 우선 말뭉치(Corpus)를 일정한 단위인 토큰(Token)으로 나눠야 한다.

말뭉치: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어. 뉴스 기사, 사용자 리뷰, 저널이나 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터. 일련의 단어의 가능성을 예측하는 알고리즘인 언어 모델을 구축하고 평가하는 데 자주 사용된다.

토큰: 개별 단어나 문장 부호와 같은 텍스트를 의미하며 말뭉치보다 작은 단위. 토큰은 텍스트의 개별 단어, 구두점 혹은 기타 의미 단위일 수 있다.

토큰화(Tokenization): 컴퓨터가 자연어를 이해할 수 있게 말뭉치를 나누는 과정.

토큰나이저(Tokenizer): 텍스트 문자열을 토큰으로 나누는 알고리즘 혹은 소프트웨어. 토큰화를 위해 사용된다. 토큰나이저를 사용해 문장을 토큰화하면 다음과 같이 된다:

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', '려', '수', '있', '다', '.']

토큰을 나누는 기준은 구축하려는 시스템이나 주어진 상황에 따라 다르다. 그리고 어떻게 토큰을 나누었느냐에 따라 시스템의 성능이나 처리 결과가 크게 달라지기도 한다.

토큰나이저를 구축하는 방법은 다음과 같다:

- 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
- 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
- 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
- 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.

어휘 사전(Vocabulary): 사전에 정의된 단어를 활용해 토큰나이저를 구축하는 방법. 직접 어휘 사전을 구축하기 때문에 없는 단어나 토큰이 존재할 수 있다. 이러한 토큰을 OOV(Out of Vocab)라고 한다. 어휘 사전 방법은 OOV 문제를 고려해 토큰화하는 것이 중요하다.

큰 어휘 사전을 구축하면 학습 비용의 증대는 물론이고, 자칫 차원의 저주(Curse of Dimensionality)에 빠질 수 있다. 모든 토큰이나 단어를 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요하고, 벡터값이 거의 모두 0의 값을 가지는 희소(sparse) 데이터로 표현된다. 또한 희소 데이터의 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려하기 때문에 문장에서 발생한 토큰들의 순서 관계를 잘 표현하지 못한다.

단어 및 글자 토큰화

토큰화 과정은 정확한 분석을 위해 필수이며, 단어나 문장의 빈도수, 출현 패턴 등을 파악할 수 있다.

또한 작은 단위로 분해된 텍스트 데이터는 컴퓨터가 이해하고 처리하기 용이해 기계 번역, 문서 분류, 감성 분석 등 다양한 자연어 처리 작업에 활용할 수 있다.

토큰화에는 단어 토큰화와 글자 토큰화가 있다.

단어 토큰화

단어 토큰화(Word Tokenization): 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나로 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업

단어 토큰화는 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 진행된다.

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고']
```

split 메서드는 주어진 구분자를 통해 문자열을 리스트 데이터로 나눠준다. 구분자를 입력하지 않으면 공백을 기준으로 나눈다.

하지만 'cg.'와 'cg'가 다른 단어로 구분되는 등, 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류 등에 취약하다.

글자 토큰화

글자 토큰화: 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식으로, 비교적 작은 단어 사전을 구축할 수 있다는 장점이 있다. 이를 통해 학습 시 컴퓨터 자원을 아낄 수 있으며, 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습할 수 있다.

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)
```

```
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', ' '
```

자모 라이브러리: 한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

컴퓨터가 한글을 인코딩하는 방식은 크게 조합형과 완성형으로 나뉜다.

조합형은 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현한다. 완성형은 조합된 글자 자체에 값을 부여해 인코딩한다.

예제 5.3 자소 단위 토큰화

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

출력 결과

```
['ㅎ', 'ㄹ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅈ', 'ㅍ', 'ㅈ', 'ㄴ', ' ', 'ㅍ', 'ㅈ'
```

단어 토큰화의 단점이었던 'cg도', 'cg는' 등에서 '도'와 같은 접사와 문장 부호의 의미를 학습할 수 있다. 또, 작은 크기의 단어 사전으로도 OOV를 획기적으로 줄일 수 있다.

하지만 개별 토큰은 아무 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 한다. 또 문장 생성이나 개체명 인식 등을 구현할 경우, 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있다. 또한, 모델 입력 시퀀스의 길이가 길어질수록 연산량이 증가한다.

형태소 토큰화

형태소 토큰화: 텍스트를 형태소 단위로 나누는 토큰화 방법. 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업. 한국어와 같은 교착어에서 중요하게 수행된다. 한국어는 다른 언어와 달리 각 단어가 띄어쓰기로 구분되지 않는다. 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이루므로 각 형태소를 적절히 구분해 처리해야 한다.

형태소(Morpheme): '그', '-는', '나', '-에게' 등과 같이 실제로 의미를 갖고 있는 최소 단위

형태소 어휘 사전

형태소 어휘 사전(Morpheme Vocabulary): 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전. 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있어 형태소 분석 작업에서 매우 중요한 역할을 한다.

일반적으로 형태소 어휘 사전에는 각 형태소가 어떤 품사에 속하는지와 해당 품사의 뜻 등의 정보도 함께 제공된다.

품사 태깅(POS Tagging): 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업. 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있어 더욱 정확한 분석이 가능해진다.

e.g. 그는 나에게 인사를 했다 -> 그(명사) + 는(조사) + 나(명사) + 에게(조사) + 인사(명사) + 를(조사) + 했다(동사)

KoNLPy, NLTK, spaCy 라이브러리를 활용해 형태소 단위의 토큰화를 실습한다.

KoNLPy

KoNLPy: 한국어 자연어 처리를 위해 개발된 라이브러리. 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공한다. 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용한다.

예제 5.4 Okt 토큰화

```
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출: ", nouns)
print("구 추출: ", phrases)
print("형태소 추출: ", morphs)
print("품사 태깅: ", pos)
```

출력 결과

```
명사 추출: ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 추출: ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출: ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어', '낼']
품사 태깅: [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'),
```

예제 5.5 꼬꼬마 토큰화

```
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출: ", nouns)
print("문장 추출: ", sentences)
print("형태소 추출: ", morphs)
print("품사 태깅: ", pos)
```

출력 결과

```
명사 추출: ['무엇', '상상', '수', '사람', '무엇']
문장 추출: ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 추출: ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이',
품사 태깅: [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ',
```



동일한 메서드라도 형태소 분석기의 특징에 따라 다른 결과가 나타난다. 예를 들어 꼬꼬마 명사 추출의 경우 Okt에서 반환하던 '낼'과 '수'를 반환하지 않았으며, 형태소 추출의 경우 '할'을 더 분리해 '하', 'ㄹ'로 반환한 것을 확인할 수 있다.

또 Okt는 총 19개의 품사로 구분해 반환하는 한편, 꼬꼬마는 더 세분화해 56개의 품사로 태깅한다. 태깅하는 품사의 수가 많으면 더 자세한 단위로 분석이 가능하지만, 품사 태깅에 소요되는 시간도 길어지며, 더 많은 품사로 분리해 모델의 성능이 저하될 수도 있다.

다음 표들은 Okt와 꼬꼬마에서 사용하는 품사 태그를 정리한 것이다:

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	SO	불임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

NLTK

NLTK(Natural Language Toolkit): 자연어 처리를 위해 개발된 라이브러리로 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능을 제공한다. 주로 영어 자연어 처리를 위해 개발됐지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공한다.

NLTK 라이브러리를 활용해 토큰화나 품사 태깅 작업을 하기 위해서는 해당 작업을 수행할 수 있는 패키지나 모델을 다운로드해야 한다. 여기서는 Punkt 모델과 Averaged Perceptron Tagger 모델을 활용해 토큰화 및 품사 태깅 작업을 수행한다. 두 모델 모두 트리뱅크(Treebank)라는 대규모 영어 말뭉치를 기반으로 학습됐다. Punkt 모델은 통계 기반 모델이며, Averaged Perceptron Tagger는 퍼셉트론을 기반으로 품사 태깅을 수행한다.


```
import nltk

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
```

예제 5.7 영문 토큰화

```
from nltk import tokenize
nltk.download('punkt_tab')

sentence = "Those who can imagine anything, can create the impossible"

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)
```

출력 결과

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible']
['Those who can imagine anything, can create the impossible']
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

영문 토큰화는 사전에 설치한 punkt 모델을 기반으로 단어나 문장을 토큰화한다. 단어 토큰나이저(word_tokenize)는 문장을 입력받아 공백을 기준으로 단어를 분리하고, 구두점 등을 처리해 각각의 단어(token)를 추출해 리스트로 반환한다.

문장 토큰나이저(sent_tokenize)는 문장을 입력받아 마침표, 느낌표, 물음표 등의 구두점을 기준으로 문장을 분리해 리스트로 반환한다.

예제 5.8 영문 품사 태깅

```
from nltk import tag
from nltk import tokenize
nltk.download('averaged_perceptron_tagger_eng')

sentence = "Those who can imagine anything, can create the impossible"

word_tokens = tokenize.word_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(pos)
```

출력 결과

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ' ',
```



NLTK 라이브러리의 품사 태깅(pos_tag) 메서드는 토큰화된 문장에서 품사 태깅을 수행한다. 품사 태깅을 수행하기 위해서는 토큰화된 단어들이 들어가야 하며, 앞서 설치한 Averaged Perceptron Tagger 모델이 설치되어 있어야 한다.

Averaged Perceptron Tagger 모델은 총 35개의 품사를 태깅할 수 있다:

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

spaCy

spaCy: 사이언 기반으로 개발된 오픈 소스 라이브러리로서, NLTK 라이브러리와 마찬가지로 자연어 처리를 위한 기능을 제공한다. NLTK와의 차이점으로는 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리라는 점이 있다.

NLTK는 학습 목적으로 자연어 처리에 대한 다양한 알고리즘과 예제를 제공하는 반면, spaCy는 효율적인 처리 속도와 높은 정확도를 제공하는 것을 목표로 한다. 그래서 NLTK에서 사용하는 모델보다 더 크고 복잡하며 더 많은 리소스를 요구한다.

spaCy는 GPU 가속을 비롯해 영어, 프랑스어, 한국어, 일본어 등 24개 이상의 언어로 사전 학습된 모델을 제공한다. 여기서는 spaCy 라이브러리를 활용해 영어를 대상으로 토큰화할 예정이므로 영어로 사전 학습된 모델인 en_core_web_sm을 설치한다.

예제 5.9 spaCy 품사 태깅

```
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible"
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

출력 결과

```
[PRON - DT ] : Those
[PRON - WP ] : who
[AUX - MD ] : can
[VERB - VB ] : imagine
[PRON - NN ] : anything
[PUNCT - , ] : ,
[AUX - MD ] : can
[VERB - VB ] : create
[DET - DT ] : the
[ADJ - JJ ] : impossible
```

spaCy는 사전 학습된 모델을 기반으로 처리하기 때문에 spaCy 모델 불러오기 함수(load)를 통해 모델을 설정할 수 있다. 모델을 불러온 다음 문장을 입력한다.

spaCy는 객체 지향적으로 구현돼 처리한 결과를 doc 객체에 저장한다. doc 객체는 여러 개의 token 객체로 이루어져 있으며, 이 token 객체에 대한 정보를 기반으로 다양한 자연어 처리 작업을 수행한다. nlp 인스턴스에 문장을 입력하면 doc 객체가 반환되며 token 객체의 tag와 pos와 같은 속성에 접근해 값을 확인할 수 있다.

다음 표는 spaCy에서 사용되는 태그를 정리한 것이다:

표 5.4 spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

띄어쓰기와 맞춤법이 잘 지켜지지 않는 경우나 신조어, 외래어, 특정 도메인에서 사용되는 축약어 등은 완벽히 대응하기 어렵다.

하위 단어 토큰화

언어는 시간이 지나면서 변화하고, 새로운 단어나 표현이 등장하며 더 이상 사용되지 않는 단어나 표현도 생긴다. 현대의 일상 언어에서는 맞춤법이나 띄어쓰기가 엄격하게 지켜지지 않는 경우가 많고, 형태소 분석기의 취약점인 신조어나 고유어 등이 빈번하게 생겨난다. 또, 시스템에서 분석하려는 텍스트 데이터는 컴퓨터나 스마트폰 등 다양한 디지털 매체에서 생성되는데, 이러한 텍스트는 오타자가 발생할 확률이 높다.

즉, 형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에 취약하며, 이는 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다. 이를 해결하기 위한 방법 중 하나로 하위 단어 토큰화(Subword Tokenization)가 있다.

하위 단어 토큰화: 하나의 단어를 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화하는 방법

하위 단어 토큰화를 적용하면 단어의 길이를 줄일 수 있어 처리 속도가 빨라질 뿐만 아니라, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있다.

하위 단어 토큰화 방법으로는 바이트 페어 인코딩, 워드피스, 유니그램 모델 등이 있다. 여기서는 바이트 페어 인코딩과 워드피스 알고리즘에 대해 알아본다.

바이트 페어 인코딩

바이트 페어 인코딩(Byte Pair Encoding, BPE): 다이그램 코딩(Digram Coding)이라고도 하며 하위 단어 토큰화의 한 종류다. 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘으로, 초기에는 데이터 압축을 위해 개발됐으나 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용된다.

이 알고리즘은 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복한다. 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다.

바이트 페어 인코딩은 입력 데이터에서 가장 많이 등장한 글자의 빈도수를 측정하고, 가장 빈도수가 높은 글자 쌍을 탐색한다.

센텐스피스

센텐스피스와 코포라 라이브러리를 활용해 토큰나이저를 학습해 본다.

센텐스피스 라이브러리는 구글에서 개발한 오픈소스 하위 단어 토큰나이저 라이브러리다. 바이트 페어 인코딩과 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성한다. 또한 워드피스, 유니코드 기반의 다양한 알고리즘을 지원하며 사용자가 직접 설정할 수 있는 하이퍼파라미터들을 제공해 세밀한 토큰나이징 기능을 제공한다.

코포라 라이브러리는 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리다.

토큰나이저 모델 학습

센텐스피스 라이브러리를 통해 바이트 페어 인코딩을 수행하는 토큰나이저 모델을 학습해 본다. 코포라 라이브러리를 통해 2017년 8월부터 2019년 3월까지 청와대 청원 게시판에 올라온 청원 말뭉치를 가져온다.

예제 5.10 청와대 청원 데이터 다운로드

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일 :", petition.begin)
print("청원 종료일 :", petition.end)
print("청원 동의 수 :", petition.num_agree)
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

출력 결과

```
청원 시작일 : 2017-08-25
청원 종료일 : 2017-09-24
청원 동의 수 : 88
청원 범주 : 육아/교육
청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.
청원 본문 : 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비
```

예제 5.11 학습 데이터셋 생성

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open("../datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

corpus의 get_all_texts 메서드로 본문 데이터셋을 한 번에 불러올 수 있다. 청원 데이터를 하나의 텍스트 파일로 저장한다.

이제 corpus.txt 파일을 활용해 센텐스피스 라이브러리의 SentencePieceTrainer로 토큰나이저 모델 학습을 진행한다.

예제 5.12 토큰나이저 모델 학습

```
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=../datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 model_type=bpe"
)
```

센텐스피스 라이브러리는 SentencePieceTrainer 클래스의 Train 메서드로 토큰나이저 모델을 학습할 수 있다. 센텐스피스는 문자열 입력을 통해 인자들을 전달받는다.

센텐스피스 라이브러리의 주요한 하이퍼파라미터:

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰나이저가 다룰 수 있는 글자의 비율

model_type	토큰나이저 알고리즘
	■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

토큰나이저 모델 학습이 완료되면 petition_bpe.model 파일과 petition.bpe.vocab 파일이 생성된다. model 파일은 학습된 토큰나이저가 저장된 파일이며, vocab 파일은 어휘 사전이 저장된 파일이다.

예제 5.13 바이트 페어 인코딩 토큰화

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토큰나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰나이저를 사용할 수 있습니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)

print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

출력 결과

```
단일 문장 토큰화 : ['__안녕하세요', ',', ' __토', '크', '나', '이', '저', '가', ' __잘', ' __학', '습', ' '
여러 문장 토큰화 : [[' __이렇게', ' __입', '력', '값을', ' __리', '스트', '로', ' __받아서'], [' __쉽게', ' __
단일 문장 정수 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787,
```

여러 문장 정수 인코딩 : `[[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544,`
정수 인코딩에서 문장 변환 : `['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']`
하위 단어 토큰에서 문장 변환 : `['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']`

센텐스피스 토큰라이저 모델은 `SentencePieceProcessor` 클래스를 통해 학습된 모델을 불러올 수 있다. 토큰라이저 모델 불러오기(`tokenizer.load`) 메서드를 통해 `petition_bpe.model` 모델을 불러온다.

`encode_as_pieces` 메서드는 문장을 토큰화하며, `encode_as_ids` 메서드는 토큰을 정수로 인코딩해 제공한다. 이 정수 데이터는 어휘 사전의 토큰에 매핑된 ID 값을 의미한다. 이 ID 값을 활용해 자연어 처리 모델을 구축한다.

토큰라이저 모델이나 자연어 처리 모델에서 나온 정수는 `decode_ids` 메서드나 `decode_pieces` 메서드를 통해 문자열 데이터로 변환할 수 있다.

예제 5.14 어휘 사전 불러오기

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size :", len(vocab))
```

출력 결과

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '___이')]
vocab size : 8000
```

`get_piece_size` 메서드는 센텐스피스 모델에서 생성된 하위 단어 개수를 반환하며, `id_to_piece` 메서드는 정숫값을 하위 단어로 변환하는 메서드다. 그러므로 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구성한다.

어휘 사전 크기는 모델 학습 시 설정한 8,000으로 학습됐다. 센텐스피스 라이브러리는 설정값에 따라 학습 방법이나 어휘 사전 크기가 달라질 수 있다.

워드피스

워드피스(Wordpiece) 토큰라이저는 바이트 페어 인코딩 토큰라이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합한다.

워드피스는 학습 과정에서 확률적인 정보를 사용한다. 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

이렇게 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 있으며, 이를 통해 모델이 좀 더 정확한 하위 단어로 분리할 수 있다. 각 글자 쌍에 대한 점수는 다음과 같이 계산된다:

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

f 는 빈도(frequency)를 나타내는 함수이며, x 와 y 는 병합하려는 하위 단어를 의미한다. 그러므로 $f(x,y)$ 는 x 와 y 가 조합된 글자 쌍의 빈도를 의미한다. 즉, xy 글자 쌍의 빈도가 된다. 그러므로 $score$ 는 x 와 y 를 병합하는 것이 적절한지를 판단하기 위한 점수가 된다.

워드피스 토큰라이저는 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습한다.

토큰라이저스

토큰라이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰라이저를 구현하고 학습할 수 있다. 이번에는 센텐스피스 라이브러리 대신 허깅 페이스의 토큰라이저스(Tokenizers) 라이브러리를 사용한다.

토큰라이저스 라이브러리는 정규화(Normalization)와 사전 토큰화(Pre-tokenization)를 제공한다.

정규화는 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업을 수행한다. 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등을 제공한다.

사전 토큰화는 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공한다. 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리하고 모델의 성능을 향상시킬 수 있다.

학습에 사용되는 말뭉치는 바이트 페어 인코딩 토큰라이저 학습에 사용된 말뭉치를 사용한다.

예제 5.15 워드피스 토큰라이저 학습

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
```

```
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())

tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["../datasets/corpus.txt"])
tokenizer.save(["../models/petition_wordpiece.json"])
```

토크나이저스 라이브러리의 토크나이저로 워드피스 모델을 불러온다. 모델을 불러왔다면 정규화 방식과 사전 토큰화 방식을 설정한다.

정규화 방식은 `normalizers` 모듈에 포함된 클래스를 불러와 시퀀스 형식으로 인스턴스를 전달한다. 이 예제에서 적용한 정규화 방식은 NFD 유니코드 정규화, 소문자 변환(Lowercase)을 사용한다.

사전 토큰화 방식도 `pre_tokenizers` 모듈에 포함된 클래스를 불러와 적용한다. 이 예제에서 적용한 사전 토큰화 방식은 공백과 구두점을 기준으로 분리한다.

정규화 방식과 사전 토큰화 방식 설정이 완료됐다면, 훈련 메서드를 통해 학습에 사용하려는 데이터셋 경로를 전달한다. 학습이 완료됐다면 저장 메서드로 학습 결과를 저장한다.

다음 표는 토크나이저스 모듈의 정규화 클래스와 사전 토큰화 클래스를 정리했다:

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _ (U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+ [\^\w\s]+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior – removed: 삭제 – isolated: 개별 토큰 처리 – merged_with_previous: 이전 문자열과 결합해 토큰 처리 – merged_with_next: 다음 문자열과 결합해 토큰 처리 – contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

워드 피스 모델의 학습이 완료됐다면 `petition_wordpiece.json` 파일이 생성된다. 학습 결과는 JSON 형태로 저장되며 정규화 및 사전 토큰화 등의 메타데이터와 함께 어휘 사전이 저장된다.

예제 5.16 워드피스 토큰화

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
```

```
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode_batch([enc.ids for enc in encoded_sentences])
```

출력 결과

```
인코더 형식 : <class 'tokenizers.Encoding'>
단일 문장 토큰화 : ['안녕하세요', ',', '토', '##크', '##나이', '##저', '##가', '잘', '학습', '##되었',
여러 문장 토큰화 : [['이렇게', '입력', '##값을', '리스트', '##로', '받아서'], ['쉽게', '토', '##크', '
단일 문장 정수 인코딩 : [8760, 11, 8693, 8415, 16269, 7536, 7488, 7842, 15016, 8670, 8734, 0]
여러 문장 정수 인코딩 : [[8187, 19643, 13834, 28119, 7495, 12607], [9739, 8693, 8415, 16269, 7536
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']
```

워드피스 토큰화를 수행하기 위해 모델 결과가 저장된 `petition_wordpiece.json` 파일을 불러와 `Tokenizer` 객체를 생성한다. 그 다음, `WordPieceDecoder()`를 사용해 `Tokenizer`의 디코더를 워드피스 디코더로 설정한다.

인코딩(`encode`) 메서드로 문장을 토큰화할 수 있으며, 인코딩 배치(`encode_batch`) 메서드로 여러 문장을 한 번에 토큰화할 수 있다. 인코딩 메서드와 인코딩 배치 메서드 모두 워드피스 토큰라이저를 통해 문장을 토큰화하고 각 토큰의 색인 번호를 반환한다.

토큰화된 데이터는 토큰(`tokens`) 속성을 통해 값을 확인할 수 있으며, 토큰 정수(`ids`) 속성으로 인코딩된 문장의 ID 값을 출력할 수 있다.

정수를 다시 문장으로 변환하는 경우 디코딩(`decode`) 메서드를 통해 정수 인코딩된 결과를 다시 문장으로 디코딩해 출력할 수 있다.

최근 연구 동향은 더 큰 말뭉치를 사용해 모델을 학습하고 OOV의 영향을 줄이기 위해 하위 단어 토큰화를 활용한다.

06 임베딩

컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 텍스트 벡터화(`Text Vectorization`) 과정이 필요하다.

기초적인 텍스트 벡터화로는 원-핫 인코딩, 빈도 벡터화(`Count Vectorization`) 등이 있다.

원-핫 인코딩: 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식

e.g. I like apples 문장과 I like bananas 문장:

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

빈도 벡터화: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

e.g. apples라는 단어가 총 4번 등장하면 해당 단어에 대한 벡터값은 4

이러한 방법은 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하다는 장점이 있지만, 벡터의 희소성이 크다는 단점이 있다.

말뭉치 내에 존재하는 토큰의 개수만큼의 벡터 차원을 가져야 하지만, 입력 문장 내에 존재하는 토큰의 수는 그에 비해 현저히 적기 때문에 컴퓨팅 비용의 증가와 차원의 저주 같은 문제를 겪을 수 있다.

이러한 문제를 해결하기 위해 워드 투 벡터(Word2Vec)나 패스트 텍스트(fastText) 등과 같이 단어의 의미를 학습해 표현하는 워드 임베딩(Word Embedding) 기법을 사용한다

워드 임베딩: 단어를 고정된 길이의 실수 벡터로 표현하는 방법으로, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론한다.

워드 임베딩은 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있다. 그래서 인공 신경망을 활용해 동적 임베딩(Dynamic Embedding) 기법을 사용한다.

언어 모델

언어 모델(Language Model): 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델. 이를 위해 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행한다.

'안녕하세요'라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어렵다. 이는 비지도 학습 문제로 모델 스스로 문장 등장 확률을 계산해야 한다. 또한, '안녕하세요' 다음에 올 수 있는 문장은 사실상 무한에 가깝다. 그러므로 완성된 문장 단위로 확률을 계산하는 것은 불가능하다. 이러한 문제를 해결하기 위해 문장 전체를 예측하는 방법 대신, 하나의 토큰 단위로 예측하는 방법인 자기회귀 언어 모델이 고안됐다.

자기회귀 언어 모델

자기회귀 언어 모델(Autoregressive Language Model): 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측한다. 즉, 언어 모델에서 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것을 의미한다.

이를 위해 이전에 등장한 모든 토큰 정보를 고려하며, 문장의 문맥 정보를 파악하여 다음 단어를 생성한다. 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, 이 과정이 반복된다.

수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

수식 6.1에 조건부 확률의 연쇄법칙을 적용한다면 수식 6.2와 같다.

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

조건부 확률의 연쇄법칙은 하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률을 이용해 계산하는 방식으로, 이전 단어들의 시퀀스가 주어졌을 때 다음에 등장하는 단어의 확률을 이전 단어들의 조건부 확률을 이용해 계산한다.

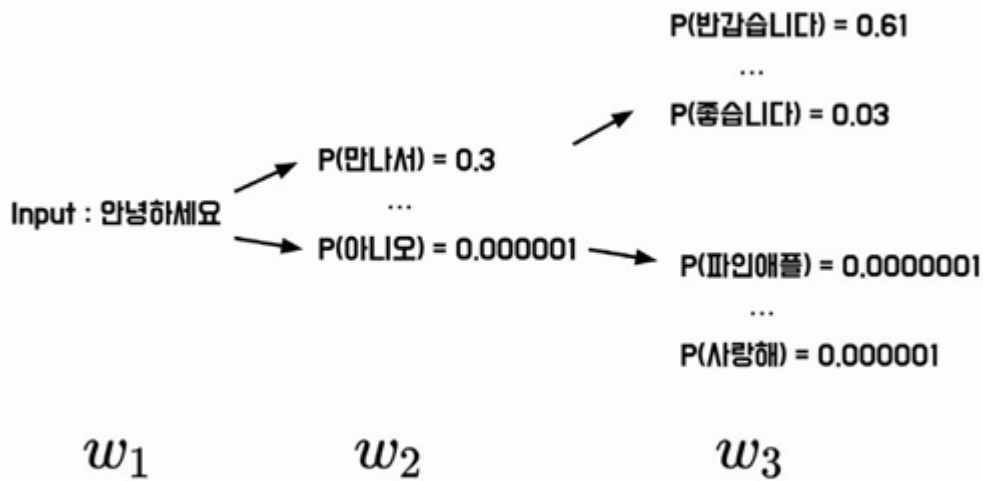


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요하다. 이를 위해 입력 토큰 w_1 을 바탕으로 모델은 다음 토큰 w_2 가 등장할 확률을 예측한다. 그다음 모델의 출력값 w_2 가 다시 입력값이 된다.

통계적 언어 모델

통계적 언어 모델(Statistical Language Model): 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측한다.

일반적으로 통계적 언어 모델은 마르코프 체인(Markov Chain)을 이용해 구현된다. 마르코프 체인은 빈도 기반의 조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측한다.

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

이 방법은 단어의 순서와 빈도에만 기초해 문장의 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있다. 또한 한 번도 등장한 적 없는 단어나 문장에 대해서는 정확한 확률을 예측하기 어렵다. 이렇게 관측한 적이 없는 데이터를 예측하지 못하는 문제를 데이터 희소성(Data sparsity) 문제라고 한다.

하지만 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로, 이를 이용해 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터를 학습할 수 있다.

통계적 언어 모델은 대규모 자연어 데이터를 처리하는 데 효과적이며, 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현할 수 있게 됐다.

최근 연구되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 사전 학습한다. 예를 들어 트랜스포머 모델에 기반한 언어 모델인 GPT나 BERT는 다음과 같은 문장 생성 기법을 이용해 모델을 사전 학습한다.

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

N-gram: 가장 기초적인 통계적 언어 모델. 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단어로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.

N-gram 모델은 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석한다. 이때, 연속된 N개의 단어를 하나의 단위로 취급하여 추론하는 모델이며, N이 1일 때는 유니그램(Unigram), N이 2일 때는 바이그램(Bigram), N이 3일 때는 트라이그램(Trigram)으로 부른다.

N이 4 이상이면 N-gram이라고 부른다.

Unigram :	<u>안녕하세요</u> 만나서 진심으로 반가워요	안녕하세요. 만나서. 진심으로. 반가워요
Bigram :	<u>안녕하세요</u> <u>만나서</u> 진심으로 반가워요	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram :	<u>안녕하세요</u> <u>만나서</u> <u>진심으로</u> 반가워요	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

N-gram은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률을 계산한다. 따라서 각 N-gram에서 t번째 토큰의 조건부 확률을 계산하는 수식은 다음과 같다.

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

이전 단어들의 개수를 결정하는 N의 값을 조정하여 N-gram 모델의 성능을 조절할 수 있다.

예제 6.1 N-gram

```
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
```

```

trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))

```

출력 결과

```

[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]

```

N-gram은 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석하는 데 효과적이다. 또한 관용적 표현 분석에도 활용된다. 자주 등장하는 연속된 단어나 구를 추출하기 때문이다.

N-gram은 시퀀스에서 연속된 n개의 단어를 추출하므로 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용된다.

TF-IDF

TF-IDF(Term Frequency-Inverse Document Frequency): 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미한다. 즉, TF-IDF는 BoW(Bag-of-Words)에 가중치를 부여하는 방법이다.

BoW: 문서나 문장을 단어의 집합으로 표현하는 방법으로, 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록한다.

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치를 얻는다.

단어 빈도

단어 빈도(Term Frequency, TF): 문서 내에서 특정 단어의 빈도수를 나타내는 값

수식 6.5 단어 빈도

$$TF(t,d) = count(t,d)$$

해당 단어의 상대적인 중요도를 측정하는 데 사용된다. 하지만 TF 값이 높다고 해당 단어가 특정 문서에서 무조건 중요한 역할을 하는 건 아니다. 단어 자체가 특정 문서 내에서 자주 사용되는 단어일 수 있다.

문서 빈도

문서 빈도(Document Frequency, DF): 한 단어가 얼마나 많은 문서에 나타나는지를 의미한다. 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수를 계산한다.

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d: d \in D)$$

DF 값이 높은 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있다. 반대로 DF 값이 낮다면 특정 단어가 적은 수의 문서에만 등장한다는 뜻이다. 그러므로 특정한 문맥에서만 사용되는 단어일 수 있으며, 중요도가 높을 수 있다.

역문서 빈도

역문서 빈도(Inverse Document Frequency, IDF): 전체 문서 수를 문서 빈도로 나눈 후 로그를 취한 값. 문서 내에서 특정 단어가 얼마나 중요한지를 나타낸다.

문서에서 특정 단어의 등장 횟수가 적으면 IDF는 상대적으로 커진다.

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

IDF는 분모의 DF 값에 1을 더한 값을 사용한다. 특정 단어가 한 번도 등장하지 않는다면 분모가 0이 되어버린다. 그러므로 1과 같은 작은 값을 더해 이를 방지한다.

추가로 IDF는 로그를 취한다. 전체 문서 수를 문서 빈도로 나눈 값은 너무 클 수 있다. 10,000개의 문서에서 특정한 단어가 1번만 등장한다면 IDF 값은 5,000이 된다.

TF-IDF

TF-IDF는 앞선 문서 빈도와 역문서 빈도를 곱한 값이다.

수식 6.8 TF-IDF

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장한다면 TF-IDF 값은 커진다. 그러므로 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어의 가중치는 낮아진다.

사이킷런 라이브러리의 TF-IDF 클래스

TF-IDF 클래스

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(  
    input="content",  
    encoding="utf-8",  
    lowercase=True,  
    stop_words=None,  
    ngram_range=(1, 1),  
    max_df=1.0,  
    min_df=1,  
    vocabulary=None,  
    smooth_idf=True,  
)
```

input: 입력될 데이터의 형태. 기본값 content는 문자열 데이터 혹은 바이트 형태의 입력값. 파일 객체를 사용한다면 file로 입력하며, 파일 경로를 사용하는 경우 filename으로 입력한다.

encoding: 바이트 혹은 파일을 입력받을 경우 사용할 텍스트 인코딩 값

lowercase: 입력받은 데이터를 소문자로 변환할지 여부. True로 설정하면 모든 입력 텍스트를 소문자로 변환한다.

stop_words: 분석에 도움이 되지 않는 의미 없는 단어들. 입력받은 단어들은 단어 사전에 추가되지 않는다.

ngram_range: 사용할 N-gram 범위. (최솟값, 최댓값) 형태로 입력하며, (1,2)는 유니그램과 바이그램을 사용한다는 것을 의미한다.

max_df: 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리한다. 정수를 입력하면 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리하며, 1 이하의 실수를 입력하면 해당 비율을 초과한 단어를 불용어 처리한다.

min_df: 전체 문서 중 일정 횟수 미만으로 등장한 단어는 불용어로 처리한다.

vocabulary: 미리 구축한 단어사전이 있다면 해당 단어 사전을 사용한다. 입력하지 않는다면 TF-IDF 학습 시 자동으로 구축된다.

smooth_idf: IDF 계산 시 분모에 1을 더한다.

예제 6.2 TF-IDF 계산


```

from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)

```

출력 결과

```

[[0.          0.          0.39687454 0.39687454 0.          0.79374908
  0.2344005 ]
 [0.61980538 0.          0.          0.          0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.          0.          0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}

```

TF-IDF 클래스는 fit 메서드를 통해 학습해야 데이터의 변환 또는 추론이 가능하다. TF-IDF 클래스를 통해 tfidf-vectorizer 객체를 불러오고, fit 메서드를 통해 준비된 말뭉치를 학습한다.

이를 통해 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있다.

하지만 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않는다. 그러므로 문장 생성과 같이 순서가 중요한 작업에는 부적합하다.

Word2Vec

Word2Vec: 2013년 구글에서 공개한 임베딩 모델로 단어 간의 유사성을 측정하기 위해 분포 가설을 기반으로 개발됐다.

분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정. 단어 간의 동시 발생 확률 분포를 이용해 단어 간의 유사성을 측정한다.

분산 표현: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것

유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 된다. 이러한 방법으로 빈도 기반 벡터화에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계를 극복했으며, 대량의

텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보를 효과적으로 표현한다.

단어 벡터화

단어를 벡터화하는 방법은 크게 희소 표현과 밀집 표현으로 나눌 수 있다. 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법은 희소 표현이며, Word2Vec은 밀집 표현이다.

표 6.3 단어의 희소 표현

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

빈도 기반 방법은 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비가 발생한다. 또한, 단어 간의 유사성을 반영하지 못하고, 벡터 간의 유사성을 계산하는 데도 많은 비용이 발생한다.

이에 비해 Word2Vec과 같은 밀집 표현은 단어를 고정된 크기의 실수 벡터로 표현하기 때문에 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않는다. 벡터 공간상에서 단어 간의 거리를 효과적으로 계산할 수 있으며, 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간을 활용할 수 있다.

밀집 표현 벡터화는 학습을 통해 단어를 벡터화하기 때문에 단어의 의미를 비교할 수 있다. 밀집 표현된 벡터를 단어 임베딩 벡터라고 한다.

Word2Vec은 밀집 표현을 위해 CBoW와 Skip-gram이라는 두 가지 방법을 사용한다.

CBoW

CBoW: 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법

중심 단어는 예측해야 할 단어를 의미하며, 예측에 사용되는 단어들을 주변 단어라고 한다.

중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 정해야 하는데, 이 범위를 윈도우(Window)라고 한다. 이 윈도우를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습한다.

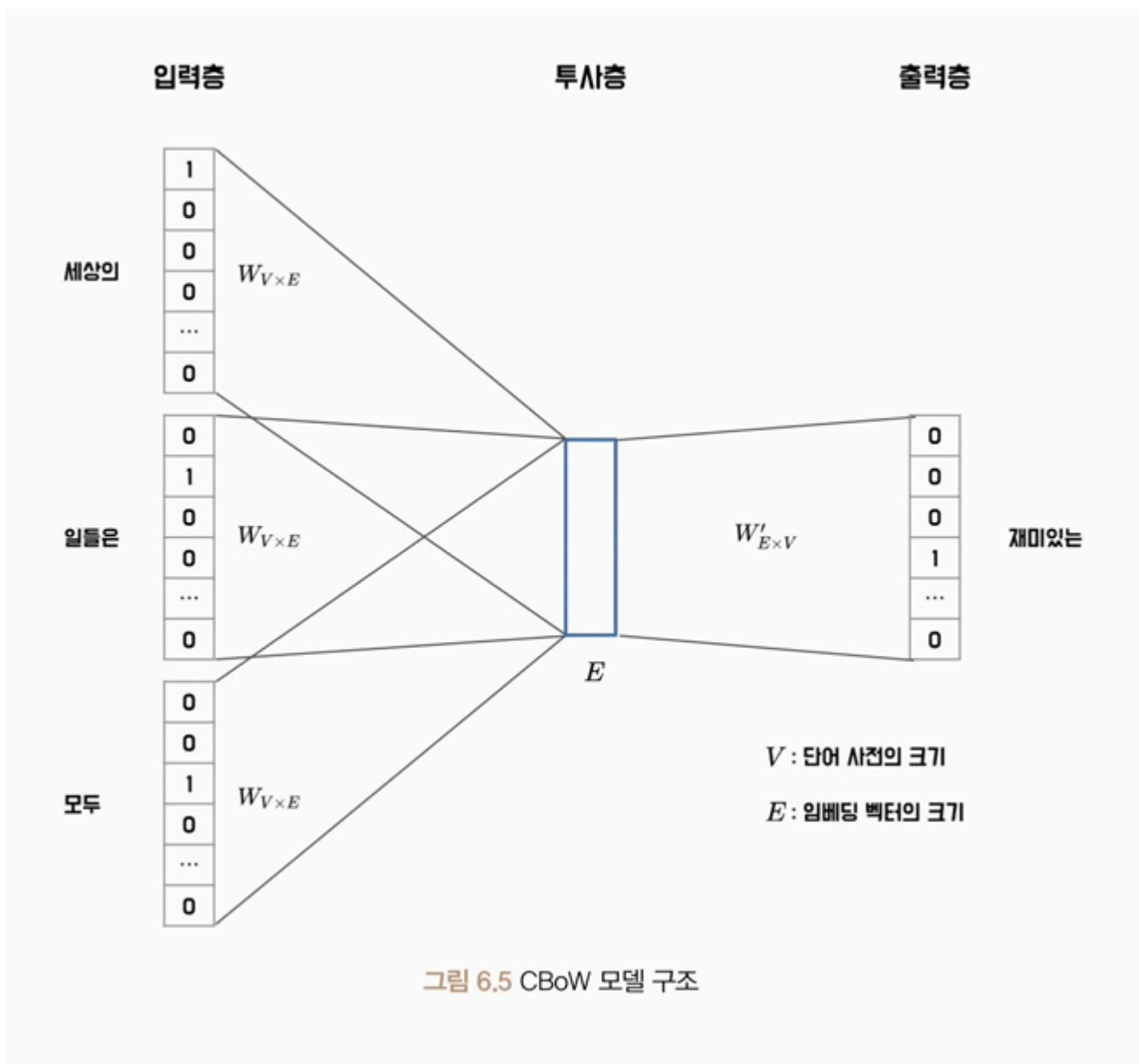
학습을 위해 윈도우를 이동해 가며 학습하는데, 이러한 방법을 슬라이딩 윈도우라고 한다. CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습할 수 있다.

다음 그림은 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터가 어떻게 구성되는지를 보여준다.

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

학습 데이터는 (주변 단어 | 중심 단어)로 구성된다. 이를 통해 대량의 말뭉치에서 효율적으로 단어의 분산 표현을 학습할 수 있다.



CBoW 모델은 각 입력 단어의 원-핫 벡터를 입력값으로 받는다. 입력 문장 내 모든 단어의 임베딩 벡터를 평균내어 중심 단어의 임베딩 벡터를 예측한다.

입력 단어는 원-핫 벡터로 표현돼 투사층에 입력된다. 투사층이란 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표 구조가 된다.

투사층을 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환된다.

계산된 평균 벡터를 가중치 행렬과 곱하면 V 크기의 벡터를 얻는다. 이 벡터에 소프트맥스 함수를 이용해 중심 단어를 예측한다.

Skip-gram

Skip-gram은 CBoW와 반대로 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델. 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만든다. 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습시킨다.

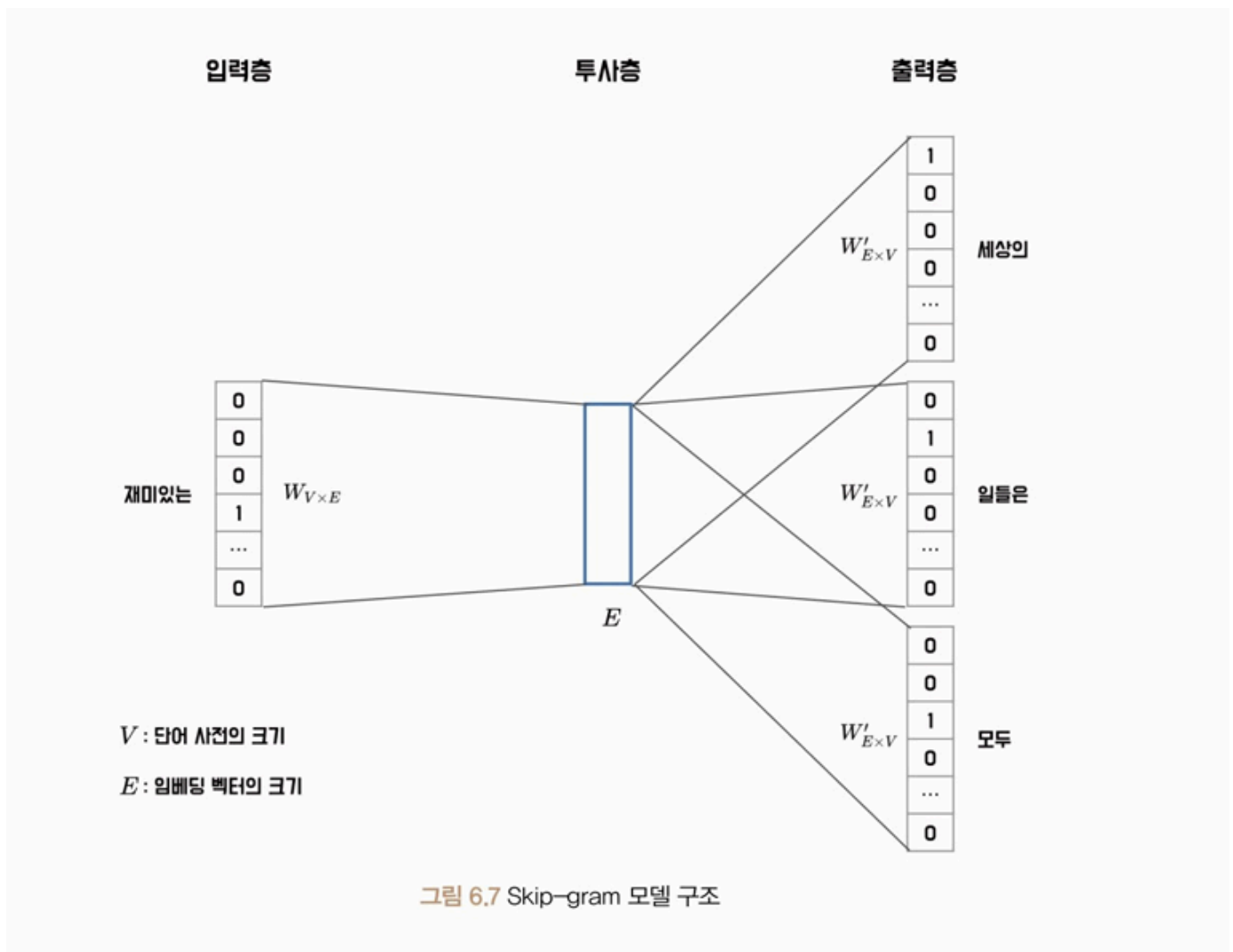
입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

Skip-gram과 CBoW는 학습 데이터 구성 방식에 차이가 있다. CBoW는 하나의 윈도우에서 하나의 학습 데이터가 만들어지는 반면 Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어진다.

이런 차이 때문에 Skip-gram은 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하므로 더 많은 학습 데이터셋을 추출할 수 있으며, 일반적으로 CBoW보다 더 뛰어난 성능을 보인다.

또한 Skip-gram은 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 되고 단어 벡터 공간에서 더 유의미한 거리 관계를 형성할 수 있다.



Skip-gram 모델도 CBoW와 마찬가지로 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져온다.

소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행한다. 말뭉치 크기가 커지면 필연적으로 단어 사전의 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 학습 속도가 느려지는 단점이 있다. 이를 해결하기 위해 계층적 소프트맥스와 네거티브 샘플링 기법을 적용해 학습 속도가 느려지는 문제를 완화한다.

계층적 소프트맥스

계층적 소프트맥스: 출력층을 이진 트리 구조로 표현해 연산을 수행한다. 자주 등장하는 단어일수록 트리의 상위 노드에 위치하고, 드물게 등장하는 단어일수록 하위 노드에 배치된다. 이런 식으로 확률을 계산하면 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성을 보인다.

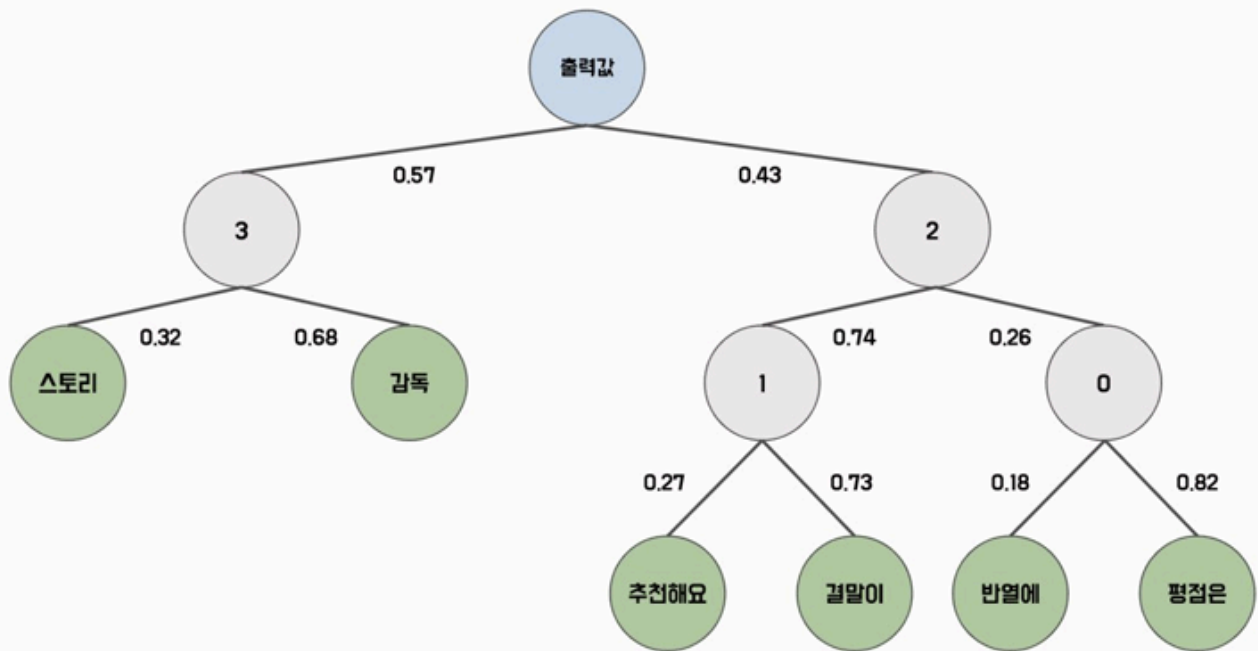


그림 6.8 계층적 소프트맥스 구조

각 노드는 학습이 가능한 벡터를 가지며, 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률을 계산한다.

앞 노드는 가장 깊은 노드로, 각 단어를 의미하며, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 한다. 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있다.

네거티브 샘플링

네거티브 샘플링: Word2Vec 모델에서 사용되는 확률적 샘플링 기법으로 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용한다.

학습 원도 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행한다. 이를 통해 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습할 수 있다.

이때 추출할 단어 n 은 일반적으로 5~20개를 사용하며, 각 단어가 추출될 확률을 다음과 같다.

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

네거티브 샘플링에서는 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 아니면 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류를 한다. 이를 위해 로지스틱 회귀 모델을 사용하며, 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습한다.

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

→

입력 데이터	출력 데이터
재미있는. 세상의	1
재미있는. 일들은	1
재미있는. 모두	1
재미있는. 걸어가다	0
재미있는. 주십시오	0
재미있는. 미리	0
재미있는. 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

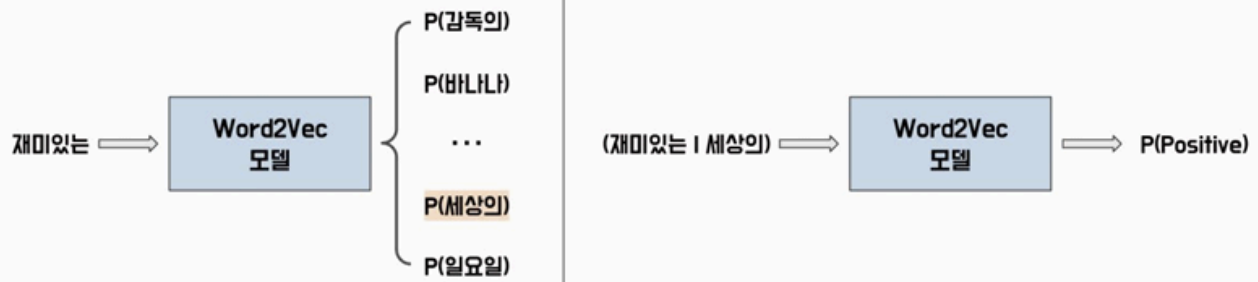


그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

네거티브 샘플링 모델에서는 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블 (0 or 1)을 가져와 내적 연산을 수행한다. 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환된다.

이때 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화된다.

모델 실습: Skip-gram

임베딩 클래스

```
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None,
    norm_type=2.0
)
```

예제 6.3 기본 Skip-gram 클래스

```
from torch import nn

class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
```

```

        num_embeddings=vocab_size,
        embedding_dim=embedding_dim
    )
    self.linear = nn.Linear(
        in_features=embedding_dim,
        out_features=vocab_size
    )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output

```

예제 6.4 영화 리뷰 데이터셋 전처리

```

import pandas as pd
from Korpora import Korpora
from konlpy.tag import Okt

corpus = Korpora.load("nsmc")
corpus = pd.DataFrame(corpus.test)

tokenizer = Okt()
tokens = [tokenizer.morphs(review) for review in corpus.text]
print(tokens[:3])

```

출력 결과

```

[['귤', 'ㅋ'], ['GDNTOPCLASSINTHECLUB'], ['뭐', '야', '이', '평점', '들', '은', '....', '나쁘진',

```

예제 6.5 단어 사전 구축

```

from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

vocab = build_vocab(corpus=tokens, n_vocab=5000, special_tokens=["<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

```

```
print(vocab[:10])
print(len(vocab))
```

출력 결과

```
['<unk>', '.', '이', '영화', '의', '...', '가', '에', '...', '을']
5001
```

예제 6.6 Skip-gram의 단어 쌍 추출

```
def get_word_pairs(tokens, window_size):
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            center_word = sentence[idx]
            context_words = sentence[window_start:idx] + sentence[idx+1:window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs

word_pairs = get_word_pairs(tokens, window_size=2)
print(word_pairs[:5])
```

출력 결과

```
[['곧', 'ㅋ'], ['ㅋ', '곧'], ['뭐', '야'], ['뭐', '아'], ['야', '뭐']]
```

예제 6.7 인덱스 쌍 변환

```
def get_index_pairs(word_pairs, token_to_id):
    pairs = []
    unk_index = token_to_id["<unk>"]
    for word_pair in word_pairs:
        center_word, context_word = word_pair
        center_index = token_to_id.get(center_word, unk_index)
        context_index = token_to_id.get(context_word, unk_index)
        pairs.append([center_index, context_index])
    return pairs

index_pairs = get_index_pairs(word_pairs, token_to_id)
print(index_pairs[:5])
```

출력 결과

```
[[595, 100], [100, 595], [77, 176], [77, 2], [176, 77]]
```

예제 6.8 데이터로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

index_pairs = torch.tensor(index_pairs)
center_indexes = index_pairs[:, 0]
context_indexes = index_pairs[:, 1]

dataset = TensorDataset(center_indexes, context_indexes)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

예제 6.9 Skip-gram 모델 준비 작업

```
from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
word2vec = VanillaSkipgram(vocab_size=len(token_to_id), embedding_dim=128).to(device)
criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(word2vec.parameters(), lr=0.1)
```

예제 6.10 모델 학습

```
for epoch in range(10):
    cost = 0.0
    for input_ids, target_ids in dataloader:
        input_ids = input_ids.to(device)
        target_ids = target_ids.to(device)

        logits = word2vec(input_ids)
        loss = criterion(logits, target_ids)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    cost += loss

    cost = cost / len(dataloader)
    print(f"Epoch : {epoch+1:4d}, Cost : {cost:.3f}")
```

출력 결과

```
Epoch :    1, Cost : 6.196
Epoch :    2, Cost : 5.980
Epoch :    3, Cost : 5.931
Epoch :    4, Cost : 5.901
```

```
Epoch :    5, Cost : 5.878
Epoch :    6, Cost : 5.861
Epoch :    7, Cost : 5.846
Epoch :    8, Cost : 5.833
Epoch :    9, Cost : 5.822
Epoch :   10, Cost : 5.812
```

예제 6.11 임베딩 값 추출

```
token_to_embedding = dict()
embedding_matrix = word2vec.embedding.weight.detach().cpu().numpy()

for word, embedding in zip(vocab, embedding_matrix):
    token_to_embedding[word] = embedding

index = 30
token = vocab[index]
token_embedding = token_to_embedding[token]
print(token)
print(token_embedding)
```

출력 결과

연기

```
[ -1.3700286  -0.53419936 -0.4979807  -0.2555711  -0.87646496 -0.43019596
   1.5725858   1.0621762   0.8001532  -0.10356946  2.5778508  -0.37419274
  -0.17541623 -0.10778847 -0.6680416  -1.6015458  -0.34703737  0.93376935
   0.7599935  -0.25358564  1.8083229   1.2734787  -0.45371383 -1.4026566
  -1.0142812  -0.56605715 -1.2538975  -0.28078136 -0.29474387 -0.73899305
  -0.8260153  -1.3297142  -0.42350522 -1.8581526  -1.0632359   0.77626246
   0.6720073  -0.33520705  0.15288073 -1.640648   -0.15055683 -1.0484744
  -0.5355086  -0.35930106 -0.7261805  -0.12321096 -0.44906664 -1.106636
   0.08471    1.2125494  -0.4449702  -0.04542563 -0.0528081   0.23515376
  -0.35838902 -0.84893566 -1.1192309   1.1554664   0.16187583  1.9194432
   0.61413866  0.37279838 -0.21057443  0.91979414 -0.4715557   0.10846171
  -0.26331607 -1.4180142   0.5514552  -1.3333958   0.82356685  0.81480145
  -1.1805079  -0.2580851   1.1900429  -1.3635898   0.03030477 -1.7568526
   0.23030454  1.7480062  -0.23844935 -1.2764553   0.03480529 -0.99042135
  -1.3860449  -0.6464405  -0.86604947 -0.50180215  0.51322734 -0.849864
   0.73899335  0.21484916 -0.3517042  -0.15372042 -1.0275272   0.9631892
  -0.00993367 -0.19878528 -0.81149644 -0.0892992  -1.7576708  -0.18071395
   0.95879394 -0.03787694  0.32819244  0.5787388  -1.3274959  -0.77856725
   1.1196953  -0.704148   -0.66877955  0.83742994 -0.94897085 -0.1719094
   0.16892032 -0.68807644  0.5901872  -1.3861064  -0.69723094 -1.3477777
   1.6631497  -0.27955115 -0.8802952  -0.8985007   0.7169123   0.81557995
   0.4633692   0.6892731  ]
```

Word2Vec 모델의 임베딩 행렬을 이용해 각 단어의 임베딩 값을 매핑하고, 인덱스 30 값의 단어와 임베딩 값을 출력한다. 이 임베딩 값으로 단어 간의 유사도를 확인할 수 있다. 임베딩의 유사도를 측정할 때는 코사인 유사도가 가장 일반적으로 사용되는 방법이다.

코사인 유사도는 두 벡터 간의 각도를 이용하여 유사도를 계산하며, 두 벡터가 유사할수록 값이 1에 가까워지고, 다를수록 0에 가까워진다.

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

a와 b는 유사도를 계산하려는 벡터이며, 두 벡터를 내적인 벡터의 크기의 곱을 나눠 코사인 유사도를 계산할 수 있다. 벡터의 크기는 각 성분의 제곱합에 루트를 씌운 값이다.

예제 6.12 단어 임베딩 유사도 계산

```
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a) / (norm(b, axis=1) * norm(a))
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1]
    return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도 : {cosine_matrix[index]:.4f}")
```

출력 결과

```
연기와 가장 유사한 5 개 단어
편집 - 유사도 : 0.3308
배우 - 유사도 : 0.3110
할듯 - 유사도 : 0.3109
주연 - 유사도 : 0.3057
높다 - 유사도 : 0.2917
```

모델 실습: Gensim

젠심 라이브러리를 활용하면 Word2Vec과 같은 자연어 처리 모델을 쉽게 구성할 수 있다. 젠심 라이브러리는 대용량 텍스트 데이터의 처리를 위한 효율적인 방법을 제공해 대규모 데이터셋에서도 효과적으로 모델을 학습할 수 있다.

Word2Vec 클래스

```
word2vec = gensim.models.Word2Vec(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000  
)
```

예제 6.13 Word2Vec 모델 학습

```
import pandas as pd  
from Korpora import Korpora  
from konlpy.tag import Okt  
  
corpus = Korpora.load("nsmc")  
corpus = pd.DataFrame(corpus.test)  
  
tokenizer = Okt()  
tokens = [tokenizer.morphs(review) for review in corpus.text]  
  
from gensim.models import Word2Vec  
  
word2vec = Word2Vec(  
    sentences=tokens,  
    vector_size=128,  
    window=5,
```

```

min_count=1,
sg=1,
epochs=3,
max_final_vocab=10000
)

word2vec.save("../models/word2vec.model")
word2vec = Word2Vec.load("../models/word2vec.model")

```

예제 6.14 임베딩 추출 및 유사도 계산

```

word = "연기"
print(word2vec.wv[word])
print(word2vec.wv.most_similar(word, topn=5))
print(word2vec.wv.similarity(w1=word, w2="연기력"))

```

출력 결과

```

[-0.06621774 -0.36674547  0.1663604   0.58909315  0.0372135  -0.02143276
  0.17337418 -0.16976535 -0.6349239   0.42311475 -0.10692125 -0.22684318
 -0.16229197 -0.05722928 -0.35821962 -0.07017373 -0.3062584   0.47364476
 -0.31980535  0.35628077  0.69763774  0.5071293  -0.32322797 -0.04212112
 -0.23542376 -0.02514695 -0.5616551   0.02662417  0.29563123  0.00791464
 -0.44979006  0.26099336  0.3812289  -0.12991887  0.09320281 -0.24161798
  0.18620354 -0.34542528 -0.2328435  -0.3889403   0.14078872  0.373868
 -0.10626898 -0.48218638 -0.28256202  0.1195678  -0.09368916 -0.3011636
  0.13934779  0.16056955  0.9585552   0.5883294  -0.00941139  0.41810927
 -0.4461008  -0.29347804  0.10102902  0.14089915 -0.14858654  0.13934498
 -0.06318886  0.08243572  0.18954007  0.19416073 -0.20150624  0.16989756
 -0.08909222  0.08483594  0.0566133  -0.12495491 -0.19386682 -0.30217433
 -0.29095522  0.14005096 -0.04750573  0.06570946 -0.21700248 -0.30987862
 -0.18673119  0.5145331   0.07081667 -0.08470267  0.21554314  0.8316306
  0.36727914  0.23544413  0.13670594 -0.33239782  0.27047485  0.06056395
 -0.03666366  0.09182108  0.1179811  -0.0013674   0.21825102 -0.10470655
 -0.1856406  -0.09957626 -0.09391525 -0.32195768 -0.1735274  -0.19358248
  0.07429539 -0.39614916 -0.03144876 -0.00670219  0.55012363  0.3297665
 -0.14430706 -0.382958   0.40008786 -0.0077886  -0.01534699  0.24336195
  0.27449661 -0.19288298  0.3035119   0.2860486  -0.00114569  0.40013826
 -0.04160194 -0.08082829  0.06571459 -0.0471548   0.01440274  0.11111417
 -0.4780807  -0.2376135 ]
[('연기력', 0.7710634469985962), ('캐스팅', 0.7409313321113586), ('몸매', 0.696537971496582), ('(
0.77106357

```

fastText

fastText: 2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델로, 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘. 단어와 문장을 벡터로 변환하는 기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용된다.

이런 벡터화 기술은 Word2Vec과 유사하지만, fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능을 제공한다. 하위 단어를 고려하기 위해 N-gram을 사용해 단어를 분해하고 벡터화하는 방법으로 동작한다.

기호가 추가된 N-gram을 사용하여 하위 단어 집합으로 분해된다.

분해된 하위 단어 집합에서 나뉘지지 않은 단어 자신도 포함되며, 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 된다.

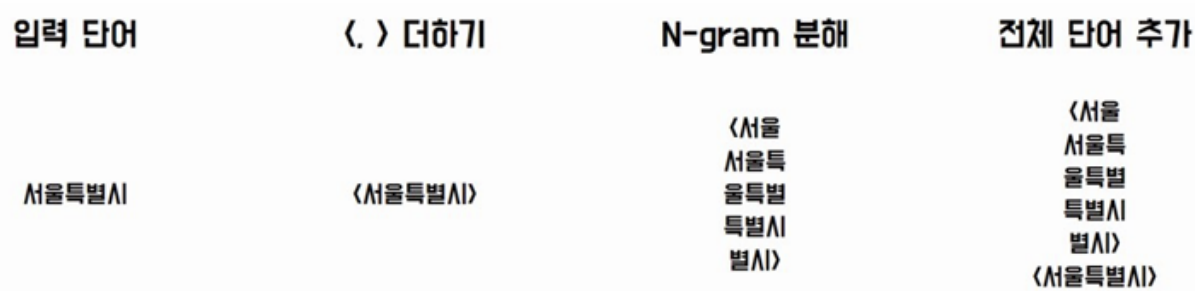


그림 6.11 fastText의 하위 단어 집합

fastText는 위와 같은 방법으로 하위 단어 집합을 생성한다. 입력 단어를 하위 단어로 분해하면 총 6개의 하위 단어가 존재하게 된다. 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산한다.

모델 실습

FastText 클래스

```
fasttext = gensim.models.FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

예제 6.15 KorNLI 데이터셋 전처리

```
from Korpora import Korpora  
  
corpus = Korpora.load("kornli")  
corpus_texts = corpus.get_all_texts() + corpus.get_all_pairs()  
tokens = [sentence.split() for sentence in corpus_texts]  
  
print(tokens[:3])
```

출력 결과

```
[['개념적으로', '크림', '스키밍은', '제품과', '자리라는', '두', '가지', '기본', '차원을', '가지고', '있고']  
[...]  
[...]]
```

예제 6.16 fastText 모델 실습

```
from gensim.models import FastText  
  
fastText = FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

```

    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=5,
    sg=1,
    max_final_vocab=20000,
    epochs=3,
    min_n=2,
    max_n=6
)

# fastText.save("../models/fastText.model")
# fastText = FastText.load("../models/fastText.model")

```

예제 6.17 fastText OOV 처리

```

oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))

```

출력 결과

```

False
[('사랑', 0.8724774122238159), ('사랑에', 0.8292666673660278), ('사랑의', 0.7907654643058777), ('

```

순환 신경망

순환 신경망: 순서가 있는 연속적인 데이터를 처리하는 데 적합한 구조를 갖고 있다. 순환 신경망은 각 시점의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동한다.

자연어 데이터는 연속적인 데이터의 일종으로 볼 수 있다.

예시는 단어들의 연속으로 이루어진 자연어 데이터가 가지는 특징을 잘 보여준다. 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추할 수 있다.

순환 신경망

순환 신경망: 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류. 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용된다.

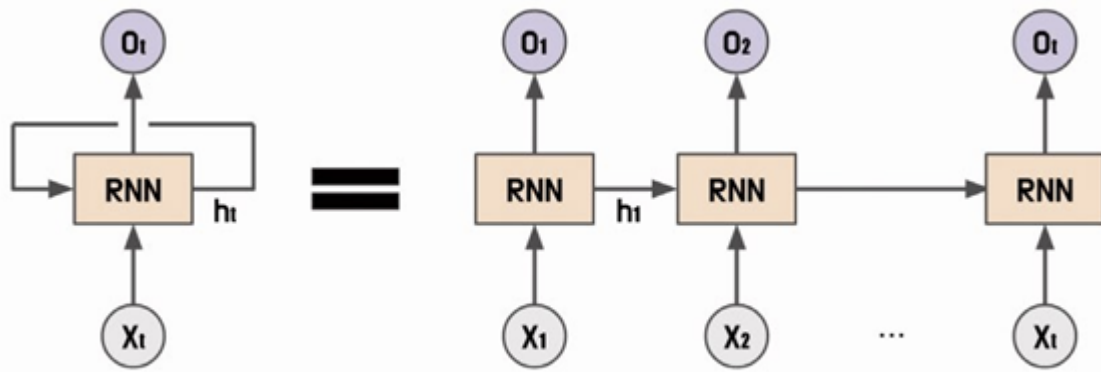


그림 6.12 순환 신경망의 셀

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

일대다 구조

일대다 구조: 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조

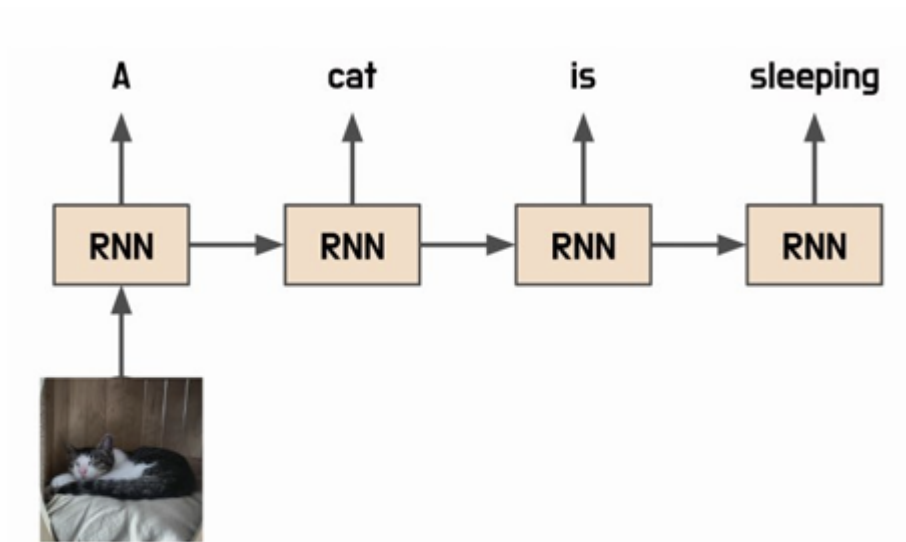


그림 6.13 순환 신경망의 일대다 구조

다대일 구조

다대일 구조: 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조

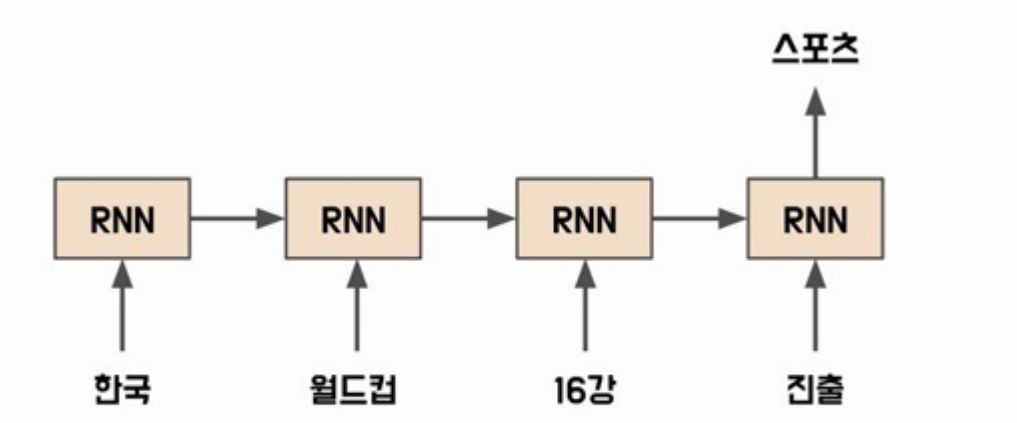


그림 6.14 순환 신경망의 다대일 구조

다대다 구조

다대다 구조: 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조

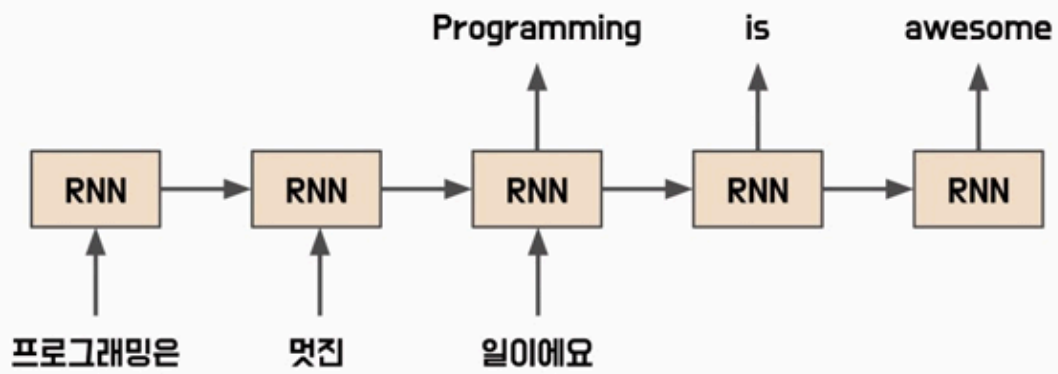


그림 6.15 순환 신경망의 다대다 구조

양방향 순환 신경망

양방향 순환 신경망: 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식

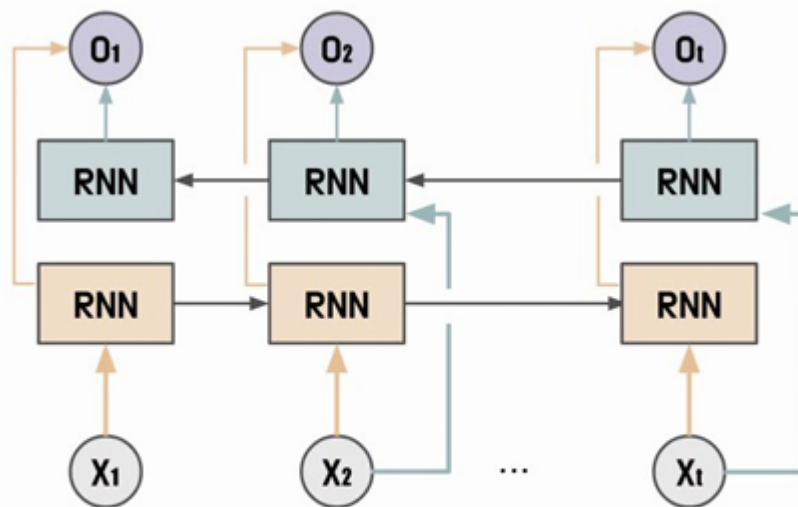


그림 6.16 양방향 순환 신경망 구조

다층 순환 신경망

다층 순환 신경망: 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계됨

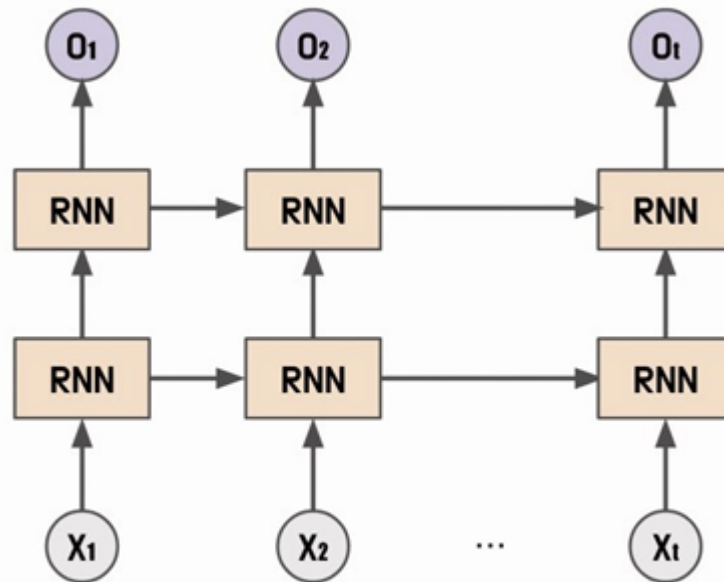


그림 6.17 다중 순환 신경망 구조

순환 신경망 클래스

순환 신경망 클래스

```

rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1,
    nonlinearity="tanh",
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False
)

```

예제 6.18 양방향 다층 신경망

```

import torch
from torch import nn

```

```

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape)
print(hidden.shape)

```

출력 결과

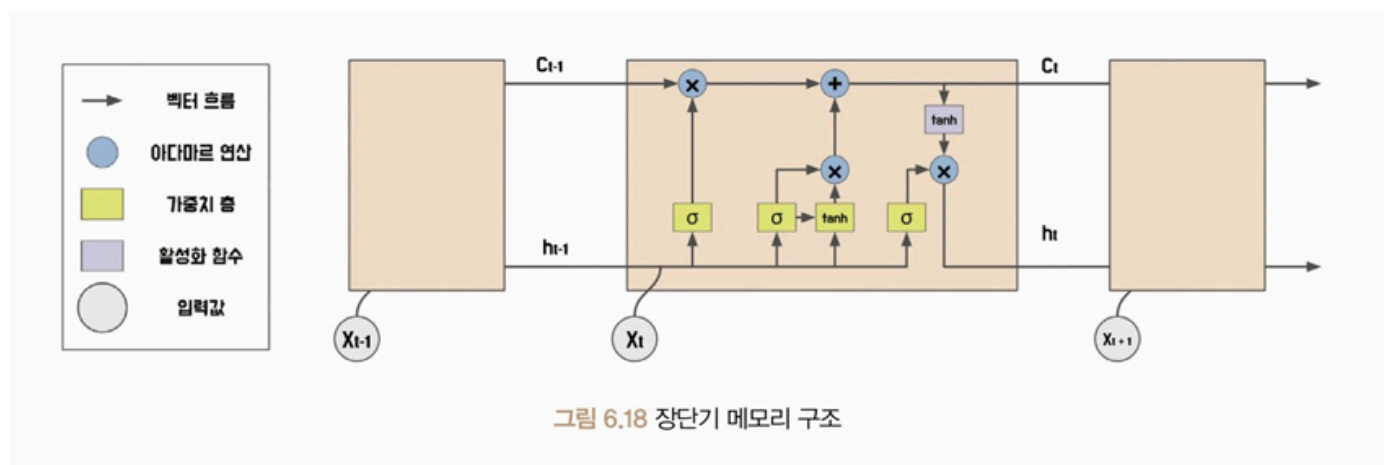
```

torch.Size([4, 6, 512])
torch.Size([6, 4, 256])

```

장단기 메모리

장단기 메모리(LSTM): 1997년 셉 호흐라이터와 유르겐 슈미트후버가 제안한 알고리즘으로 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 문제. 이를 해결하기 위해 장단기 메모리를 사용한다. 장단기 메모리는 순환 신경망과 비슷한 구조를 가지지만, 메모리 셀과 게이트라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제를 해결한다.



장단기 메모리 구조

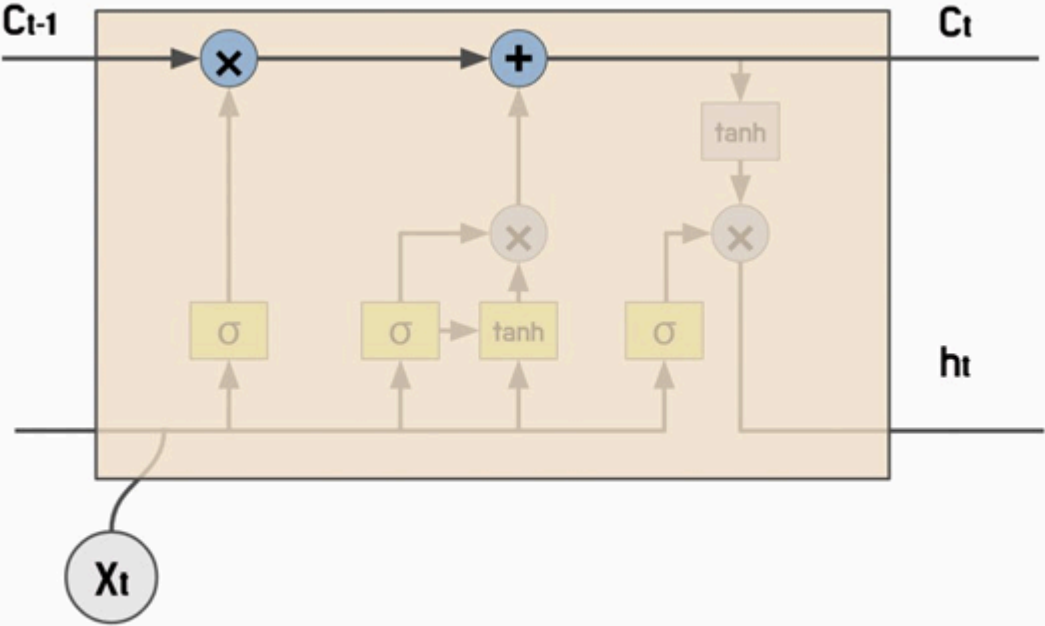


그림 6.19 장단기 메모리 셀 상태

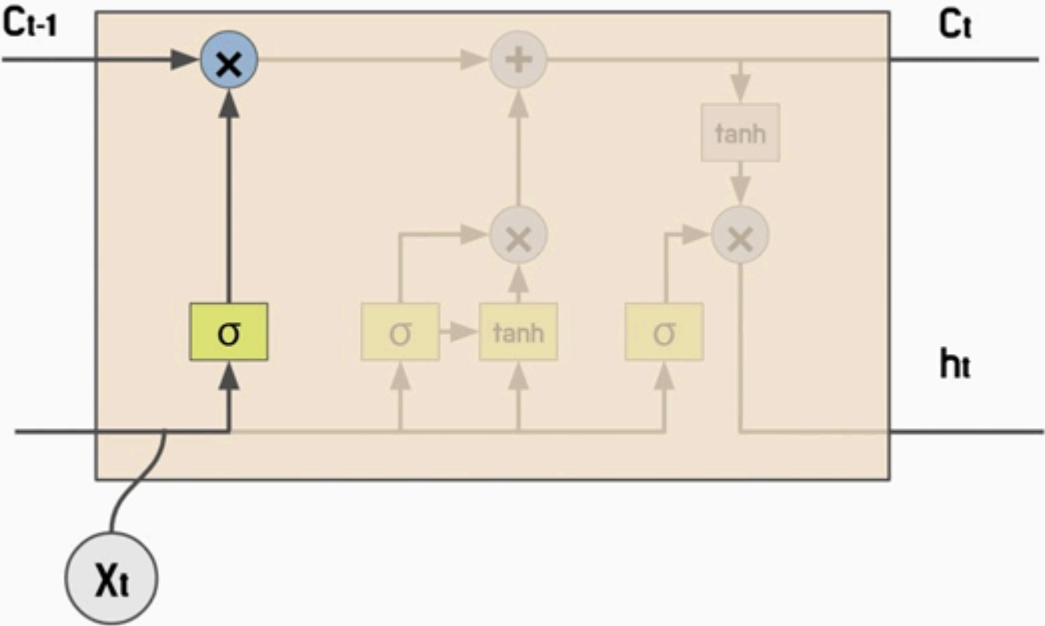


그림 6.20 장단기 메모리의 망각 게이트

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

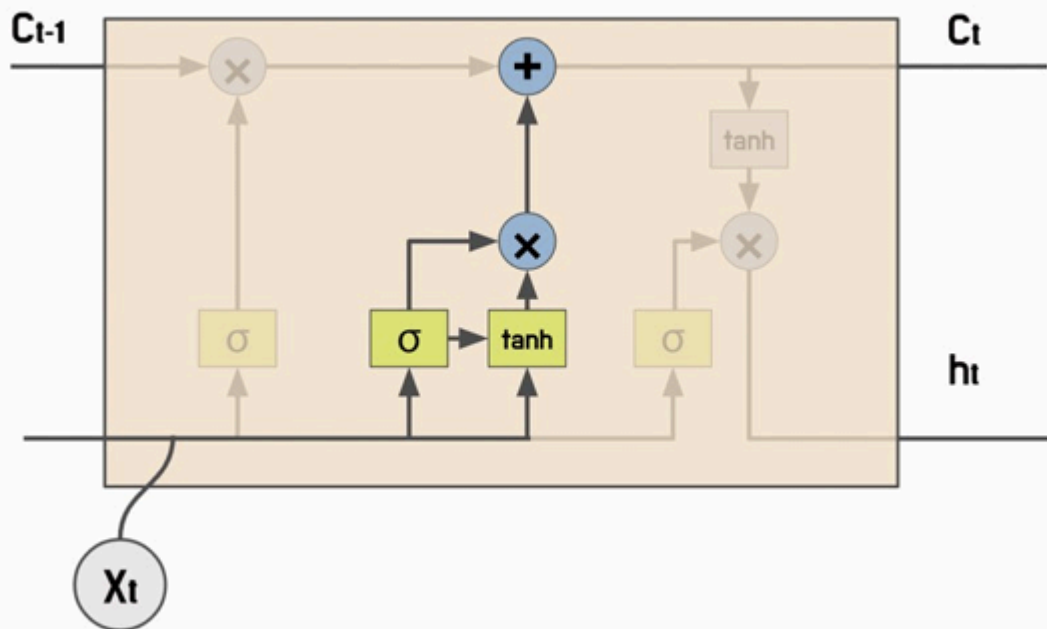


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_t \odot i_t$$

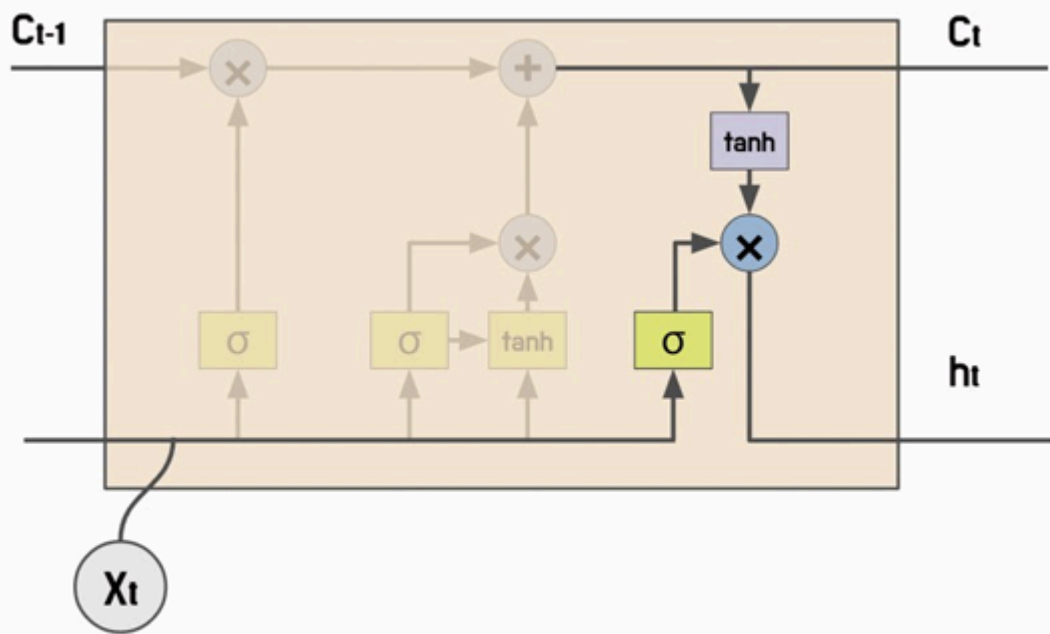


그림 6.22 장단기 메모리의 출력 게이트

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

장단기 메모리 클래스

장단기 메모리 클래스

```
lstm = torch.nn.LSTM(  
    input_size,  
    hidden_size,  
    num_layers=1,  
    bias=False,  
    batch_first=True,  
    dropout=0,  
    bidirectional=False,  
    proj_size=0  
)
```

예제 6.19 양방향 다층 장단기 메모리

```
import torch  
from torch import nn  
  
input_size = 128  
output_size = 256  
num_layers = 3  
bidirectional = True  
proj_size = 64  
  
model = nn.LSTM(  
    input_size=input_size,  
    hidden_size=output_size,  
    num_layers=num_layers,  
    batch_first=True,  
    bidirectional=bidirectional,  
    proj_size=proj_size,  
)  
  
batch_size = 4
```

```

sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else output_size,
)
c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

print(outputs.shape)
print(h_n.shape)
print(c_n.shape)

```

출력 결과

```

torch.Size([4, 6, 128])
torch.Size([6, 4, 64])
torch.Size([6, 4, 256])
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/rnn.py:1141: UserWarning: LSTM with p
result = _VF.lstm(

```

모델 실습

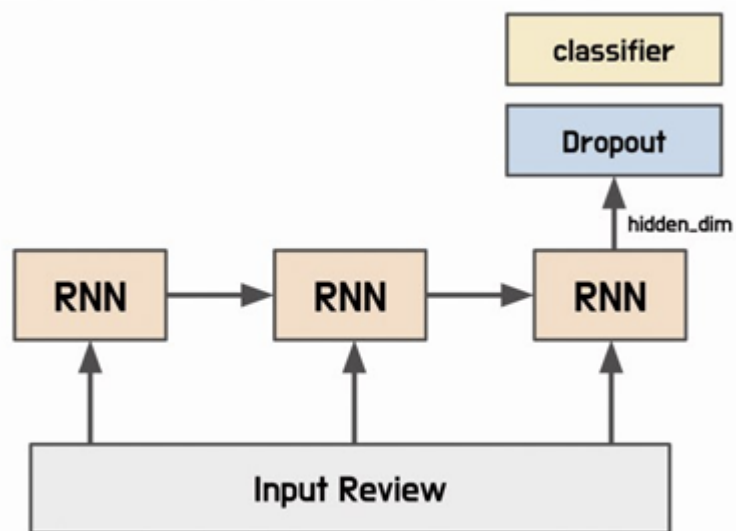


그림 6.23 긍정/부정 분류 모델 구조

```

from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm"
    ):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )
        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        if bidirectional:
            self.classifier = nn.Linear(hidden_dim * 2, 1)
        else:
            self.classifier = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        output, _ = self.model(embeddings)
        last_output = output[:, -1, :]
        last_output = self.dropout(last_output)
        logits = self.classifier(last_output)
        return logits

```

```
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))
```

출력 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 **description** 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: http://ai.stanford.edu/~amaas/data/sentiment/

# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_train.txt
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_test.txt
```

```
|      | text
|-----|:-----
| 33553 | 모든 편견을 날려 버리는 가슴 따뜻한 영화. 로버트 드 니로, 필립 세이모어 호프만 영원하라. |
|  9427 | 무한 리메이크의 소재. 감독의 역량은 항상 그 자리에... |
|   199 | 신날 것 없는 애니.
| 12447 | 잔잔 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
Training Data Size : 45000
Testing Data Size : 5000
```

예제 6.22 데이터 토큰화 및 단어 사전 구축

```
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

출력 결과

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
5002
```

예제 6.23 정수 인코딩 및 패딩

```
import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]
```



```
]
```

```
max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])
```

출력 결과

```
[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]
```

예제 6.24 데이터로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

예제 6.25 손실 함수와 최적화 함수 정의

```
from torch import optim

n_vocab = len(token_to_id)
hidden_dim = 64
embedding_dim = 128
n_layers = 2

device = "cuda" if torch.cuda.is_available() else "cpu"
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim, n_layers=n_layers
).to(device)
```

```
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)
```

예제 6.26 모델 학습 및 테스트

```
def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if step % interval == 0:
            print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits)>.5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)
```

출력 결과

```

Train Loss 0 : 0.6866534948348999
Train Loss 500 : 0.6823862150995555
Train Loss 1000 : 0.6510271433885996
Train Loss 1500 : 0.6200004250585517
Train Loss 2000 : 0.5973543037419793
Train Loss 2500 : 0.5795147728021505
Val Loss : 0.46141017963901493, Val Accuracy : 0.7792
Train Loss 0 : 0.33064547181129456
Train Loss 500 : 0.4490690616492978
Train Loss 1000 : 0.4407945908643268
Train Loss 1500 : 0.44122220700399467
Train Loss 2000 : 0.43353727353789817
Train Loss 2500 : 0.4303044144038866
Val Loss : 0.42143663623557687, Val Accuracy : 0.8098
Train Loss 0 : 0.3195874094963074
Train Loss 500 : 0.3698648054323748
Train Loss 1000 : 0.37224400674665603
Train Loss 1500 : 0.3698391514438776
Train Loss 2000 : 0.37033938994442206
Train Loss 2500 : 0.37080571157593384
Val Loss : 0.399040534235418, Val Accuracy : 0.8164
Train Loss 0 : 0.2371215522289276
Train Loss 500 : 0.3325901468565126
Train Loss 1000 : 0.3309891494539949
Train Loss 1500 : 0.33518812008733195
Train Loss 2000 : 0.33802291982505156
Train Loss 2500 : 0.3367714855860158
Val Loss : 0.3956116630722539, Val Accuracy : 0.8206
Train Loss 0 : 0.2683178782463074
Train Loss 500 : 0.29978999038210175
Train Loss 1000 : 0.29925538034929144
Train Loss 1500 : 0.30125387554999433
Train Loss 2000 : 0.30419627348634076
Train Loss 2500 : 0.30463439633981176
Val Loss : 0.4044163741718847, Val Accuracy : 0.821

```

예제 6.27 학습된 모델로부터 임베딩 추출

```

token_to_embedding = dict()
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()

for word, emb in zip(vocab, embedding_matrix):
    token_to_embedding[word] = emb

token = vocab[1000]
print(token, token_to_embedding[token])

```

출력 결과

```

보고싶다 [ 0.3060669  0.405856 -0.6062377  0.26378956 -0.49522546  0.1393255
-1.0710139  1.7116443  0.19716448 -0.27249926  1.4898953 -0.9961566
-0.57864934  0.5247539  0.3181274  0.85601634 -0.3267693 -0.15991402
 1.4392368  0.7507427  0.6704012 -1.6193398  0.05843502  2.1680667
 0.27619565  0.35380408  0.7119952 -0.2312717 -1.2261006 -0.48431394

```

```

-0.6124159  0.8361179 -0.16714838 -1.818825  -0.01270011  1.4118593
 1.4019202 -3.304726  0.645091  1.5672307 -0.024147 -0.29292262
 2.2480469 -1.9909304 -0.1897206 -0.05372866  0.7788252 -0.74750024
 0.5640589  0.37150913  0.51749355  0.6654714 -0.06851145  0.07796954
 1.6684883  1.5484782 -1.0300803 -0.00962836  0.3480971 -0.30097073
 0.62488735 -2.515338  0.5363361  0.18829024  2.2370698 -0.6247874
-1.2965727  0.97360224 -0.9887784 -0.21572527 -0.82928735 -3.0779815
-0.10086456 0.27960685 -0.45951498 0.61698633 0.05810185 -0.3953985
-0.69636995 1.0007434 -0.2281954 -1.1285896 0.3969117 -0.5400716
 1.4059969 0.7892443 0.611267 0.28321654 1.3453951 0.88394874
 1.3218311 0.83737385 0.6385697 -2.2994955 0.716826 -0.8297346
-0.9310922 0.8405451 -0.03638813 -0.56758803 -0.8356645 -1.4194324
-0.37329882 1.0024283 -1.1949705 -0.743632 0.5874541 0.32547578
 0.2913848 -0.14602208 1.1510195 -1.9079098 -0.4842883 0.24554822
 0.21113078 -0.04248791 -0.23589022 -0.9171441 -0.5523578 1.8296788
-0.2447228 0.4293891 -1.2398363 -2.067702 -0.01413966 0.7759506
 0.09090654 -0.45518965]

```

예제 6.29 사전 학습된 임베딩 계층 적용

```

import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm",
        pretrained_embedding=None
    ):
        super().__init__()
        if pretrained_embedding is not None:
            self.embedding = nn.Embedding.from_pretrained(
                torch.tensor(pretrained_embedding, dtype=torch.float32)
            )
        else:
            self.embedding = nn.Embedding(
                num_embeddings=n_vocab,
                embedding_dim=embedding_dim,
                padding_idx=0
            )

        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,

```

```

    )
elif model_type == "lstm":
    self.model = nn.LSTM(
        input_size=embedding_dim,
        hidden_size=hidden_dim,
        num_layers=n_layers,
        bidirectional=bidirectional,
        dropout=dropout,
        batch_first=True,
    )

    if bidirectional:
        self.classifier = nn.Linear(hidden_dim * 2, 1)
    else:
        self.classifier = nn.Linear(hidden_dim, 1)
    self.dropout = nn.Dropout(dropout)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    output, _ = self.model(embeddings)
    last_output = output[:, -1, :]
    last_output = self.dropout(last_output)
    logits = self.classifier(last_output)
    return logits

```

예제 6.30 사전 학습된 임베딩을 사용한 모델 학습

```

from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim,
    n_layers=n_layers, pretrained_embedding=init_embeddings
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

출력 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 **description** 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39
```

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: <http://ai.stanford.edu/~amaas/data/sentiment/>

```
# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_train.txt
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_test.txt
```

```
|      | text
|-----:|:-----
| 33553 | 모든 편견을 날려 버리는 가슴 따뜻한 영화. 로버트 드 니로, 필립 세이모어 호프만 영원하라. |
|  9427 | 무한 리메이크의 소재. 감독의 역량은 항상 그 자리에...
|   199 | 신날 것 없는 애니.
| 12447 | 잔잔 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
```

Training Data Size : 45000

Testing Data Size : 5000

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
```

5002

```
[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]
```

Train Loss 0 : 0.7048603892326355

```
/tmp/ipykernel_1395/449981664.py:87: UserWarning: To copy construct from a tensor, it is recom
train_ids = torch.tensor(train_ids)
```

```
/tmp/ipykernel_1395/449981664.py:88: UserWarning: To copy construct from a tensor, it is recom
test_ids = torch.tensor(test_ids)
```

Train Loss 500 : 0.6927574432776598

Train Loss 1000 : 0.657947656038877

Train Loss 1500 : 0.6163621224577153

Train Loss 2000 : 0.5887107210329686

Train Loss 2500 : 0.5680219356666227

Val Loss : 0.47720152615739136, Val Accuracy : 0.7822

Train Loss 0 : 0.3823622167110443

Train Loss 500 : 0.4712985908378384

Train Loss 1000 : 0.4719334789714613

Train Loss 1500 : 0.473223712019806

Train Loss 2000 : 0.4686457407751541

Train Loss 2500 : 0.46839252571578693

Val Loss : 0.4333950311135941, Val Accuracy : 0.7992

Train Loss 0 : 0.2568286061286926

Train Loss 500 : 0.46342676402090077

```
Train Loss 1000 : 0.4606587705197749
Train Loss 1500 : 0.4526384821401128
Train Loss 2000 : 0.4522544730832611
Train Loss 2500 : 0.44964724538422546
Val Loss : 0.48321028963064616, Val Accuracy : 0.7472
Train Loss 0 : 0.5696015357971191
Train Loss 500 : 0.44430506746925996
Train Loss 1000 : 0.43935377033916745
Train Loss 1500 : 0.44222024474280586
Train Loss 2000 : 0.436171752979224
Train Loss 2500 : 0.4362681849134631
Val Loss : 0.45890403836489485, Val Accuracy : 0.7604
Train Loss 0 : 0.504827618598938
Train Loss 500 : 0.4312875662110523
Train Loss 1000 : 0.42549178704664065
Train Loss 1500 : 0.42183705304143115
Train Loss 2000 : 0.4230184424435836
Train Loss 2500 : 0.42796811345266084
Val Loss : 0.41058064602053584, Val Accuracy : 0.812
```

합성곱 신경망

합성곱 신경망: 주로 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 해석하기 위해 사용되는
인공 신경망의 한 종류

합성곱 계층 클래스

2차원 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0,
    dilation=1,
    groups=1,
    bias=True,
    padding_mode="zeros"
)
```

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$

$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

활성화 맵

활성화 맵: 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값

풀링



그림 6.27 최댓값 풀링

풀링 클래스

2차원 최대값 풀링 클래스

```
pool = torch.nn.MaxPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    dilation=1  
)
```

2차원 평균값 풀링 클래스

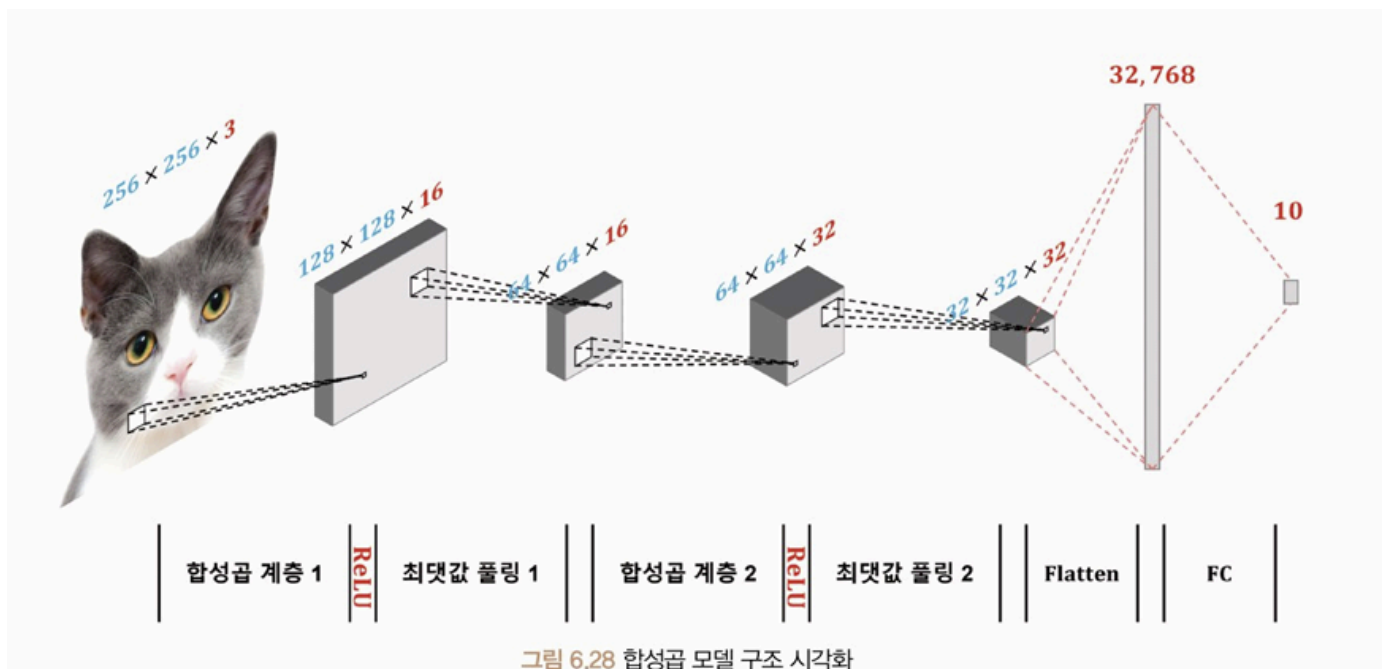
```
pool = torch.nn.AvgPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    count_include_pad=True  
)
```

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층

완전 연결 계층: 각 입력 노드가 모든 출력 노드와 연결된 상태



예제 6.31 합성곱 모델

```
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.fc = nn.Linear(32 * 32 * 32, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = torch.flatten(x)
```

```
x = self.fc(x)
return x
```

모델 실습

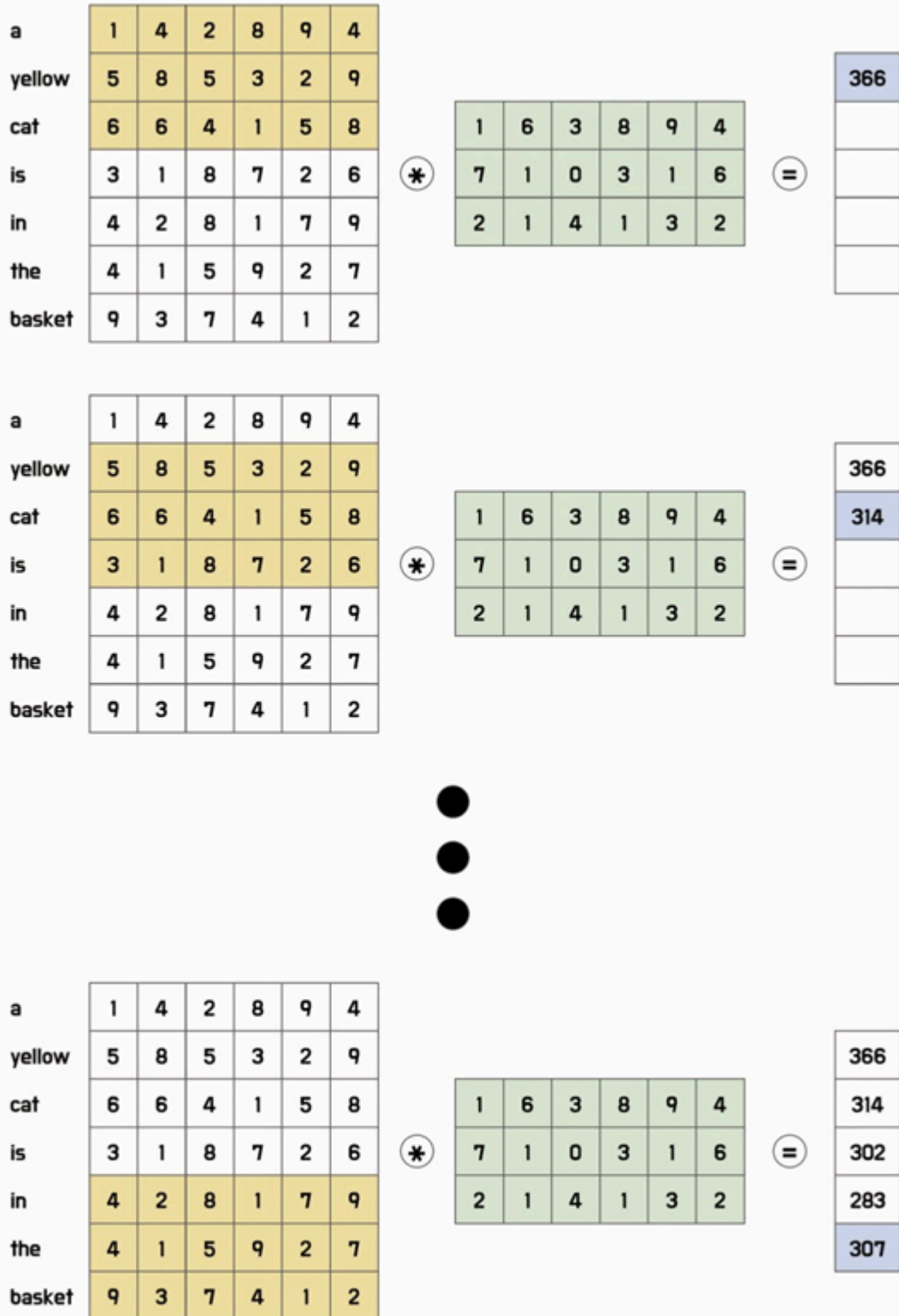


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

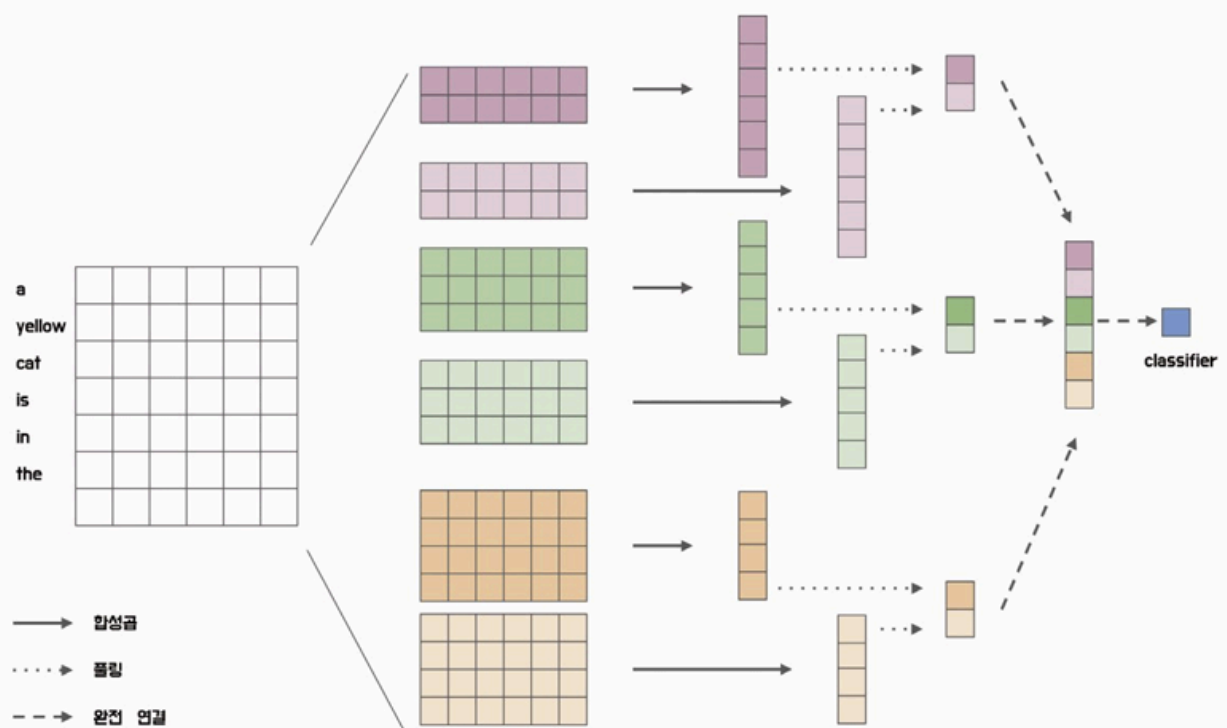


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산

예제 6.32 합성곱 기반 문장 분류 모델 정의

```
import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(self, pretrained_embedding, filter_sizes, max_length, dropout=0.5):
        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
        embedding_dim = self.embedding.weight.shape[1]

        conv = []
        for size in filter_sizes:
            conv.append(
                nn.Sequential(
                    nn.Conv1d(
                        in_channels=embedding_dim,
                        out_channels=1,
                        kernel_size=size
                    ),
                    nn.ReLU(),
```

```

        nn.MaxPool1d(kernel_size=max_length-size-1),
    )
)
self.conv_filters = nn.ModuleList(conv)

output_size = len(filter_sizes)
self.pre_classifier = nn.Linear(output_size, output_size)
self.dropout = nn.Dropout(dropout)
self.classifier = nn.Linear(output_size, 1)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    embeddings = embeddings.permute(0, 2, 1)

    conv_outputs = [conv(embeddings) for conv in self.conv_filters]
    concat_outputs = torch.cat([conv.squeeze(-1) for conv in conv_outputs], dim=1)

    logits = self.pre_classifier(concat_outputs)
    logits = self.dropout(logits)
    logits = self.classifier(logits)
    return logits

```

예제 6.33 합성곱 신경망 분류 모델 학습

```

import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))

from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

```

```

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))

import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])

import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

```

```

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

from gensim.models import Word2Vec

n_vocab = len(token_to_id)
embedding_dim = 128

word2vec = Word2Vec.load("../models/word2vec.model")
init_embeddings = np.zeros((n_vocab, embedding_dim))

for index, token in id_to_token.items():
    if token not in ["<pad>", "<unk>"]:
        init_embeddings[index] = word2vec.wv[token]

embedding_layer = nn.Embedding.from_pretrained(
    torch.tensor(init_embeddings, dtype=torch.float32)
)

from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
filter_sizes = [3, 3, 4, 4, 5, 5]
classifier = SentenceClassifier(
    pretrained_embedding=init_embeddings,
    filter_sizes=filter_sizes,
    max_length=max_length
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if step % interval == 0:

```

```

        print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

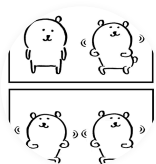
        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits)>.5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```



카사바

안녕하세요 :)



이전 포스트

Week6_예습과제

0개의 댓글

댓글을 작성하세요

댓글 작성

